



Széchenyi István Katolikus Technikum és Gimnázium

Szoftverfejlesztő és -tesztelő projektfeladat

„QUICKTOW AUTÓMENTŐK”

Készítették:

Vas Lőrinc,

Ficzere Noel Dominik

Ózd, 2026

Tartalom

1.	Bevezetés	3
2.	Felhasználói dokumentáció	4
a)	Rendszerkövetelmény	4
i)	Hardver	4
ii)	Szoftver	4
b)	Webalkalmazás indítása	4
c)	Webalkalmazás használata	5
i)	Általános információk a webalkalmazásról	5
ii)	Főoldal leírása	5
iii)	Profilbeállítások oldal leírása	9
iv)	Rólunk oldal leírása	10
3.	Fejlesztői dokumentáció	11
a)	Alkalmazás fejlesztői és csoportmunka eszközök	11
b)	Adatbázis	11
c)	Frontend	11
d)	Backend	11
4.	Adatbázis	12
5.	Frontend	17
6.	Backend	42
7.	Tesztelés	53
8.	Továbbfejlesztési lehetőségek	65
9.	Irodalomjegyzék	66

1. Bevezetés

A QuickTow webalkalmazás egy modern platformot kínál az autómentő szolgáltatások digitalizálására. A rendszer lényege, hogy a felhasználók egyszerűen és gyorsan kezdeményezhetnek autómentési / vontatási igényeket, miközben az autómentő szolgáltatók szélesebb ügyfélkört érhetnek el.

Automatikus vagy manuális helymeghatározás alapján az alkalmazás azonosítja a környéken elérhető és rendelkezésre álló autómentőket, akik közül a felhasználó választhat. Az autómentők értesítést kapnak a beérkező megrendelésekről, melyeket elfogadhatnak vagy elutasíthatnak. Elfogadás esetén a felhasználó nyomon követheti az autómentő állapotát illetve pozícióját.

A felhasználónak lehetősége van értékelést hagyni a szolgáltatásra. Ezek az értékelések megtekinthetők a mentés kérelmezése közben, az autómentő kiválasztásakor, a könnyebb döntés érdekében.

A témaválasztás során fontos szempont volt egy olyan probléma megoldása, amely a mindennapi életben gyakran előfordul, ugyanakkor jelenleg nem rendelkezik korszerű digitális megoldással.

GitHub: <https://github.com/vlorinc06/QuickTow>

2. Felhasználói dokumentáció

a) Rendszerkövetelmény

i) Hardver

Minimum hardverkövetelmény a webalkalmazás futtatásához:

- 4 GB RAM
- Kétmagos processzor
- Internetkapcsolat
- Windows 10/11

ii) Szoftver

Szükséges szoftverek a webalkalmazás futtatásához:

- XAMPP
- NodeJS
- Composer
- Angular CLI
- Laravel
- Modern webböngésző (pl. Google Chrome, Mozilla Firefox)

b) Webalkalmazás indítása

A webalkalmazás indításához először a backendet kell elindítani, amihez a következő szoftverek telepítése szükséges: XAMPP, NodeJS, Composer, Laravel. Ez után a XAMPP alkalmazásban el kell indítani az Apache és MySQL modult. Ezt követően parancssorban a backend projekt mappájába navigálva a „php artisan migrate --seed” paranccsal létre kell hozni és fel kell tölteni az adatbázist. Ez a lépés csak az első indításnál szükséges. Ez után a „php artisan serve” paranccsal elindítható a backend szerver.

A frontend indításához egy új parancssorban a frontend projekt mappájába kell navigálni, majd az „npm install” parancs kiadásával telepíteni kell a szükséges csomagokat (ez szintén

csak első indításkor szükséges). Ezután az „ng s -o” paranccsal indítható el a frontend alkalmazás.

c) Webalkalmazás használata

i) Általános információk a webalkalmazásról

A QuickTow egy webalapú alkalmazás, amely egyoldalas alkalmazásként (Single Page Application) működik, így a felhasználók számára gyors és folyamatos élményt biztosít az oldalak közötti váltás során. A rendszer két fő felhasználói típust különböztet meg: normál felhasználókat, akik vontatási kéréseket hozhatnak létre, valamint autómentő szolgáltatókat, akik ezeket a kéréseket fogadhatják és teljesíthetik.

A webalkalmazás főbb funkciói:

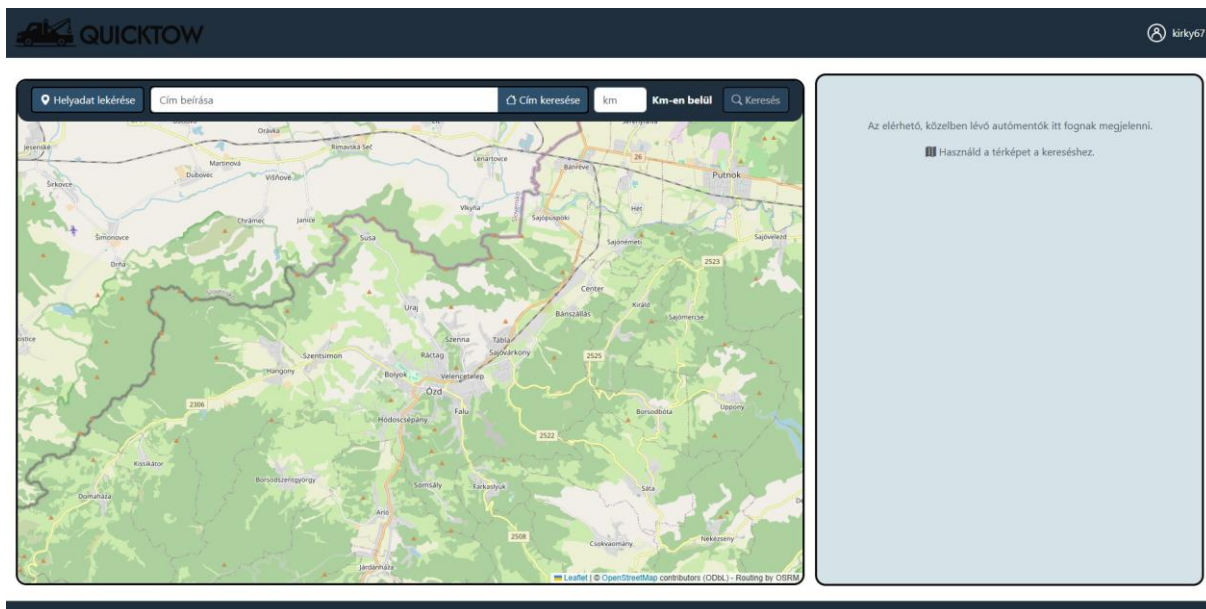
- Autómentők keresése térképes felületen
- Vontatási kérelmek létrehozása és kezelése
- Állapotkövetés
- Értékelési rendszer

ii) Főoldal leírása

A főoldalon bejelentkezés előtt egy rövid bemutató látható az oldalról, illetve az elhomályosított térképen találhatóak a bejelentkezés és regisztráció gombok.

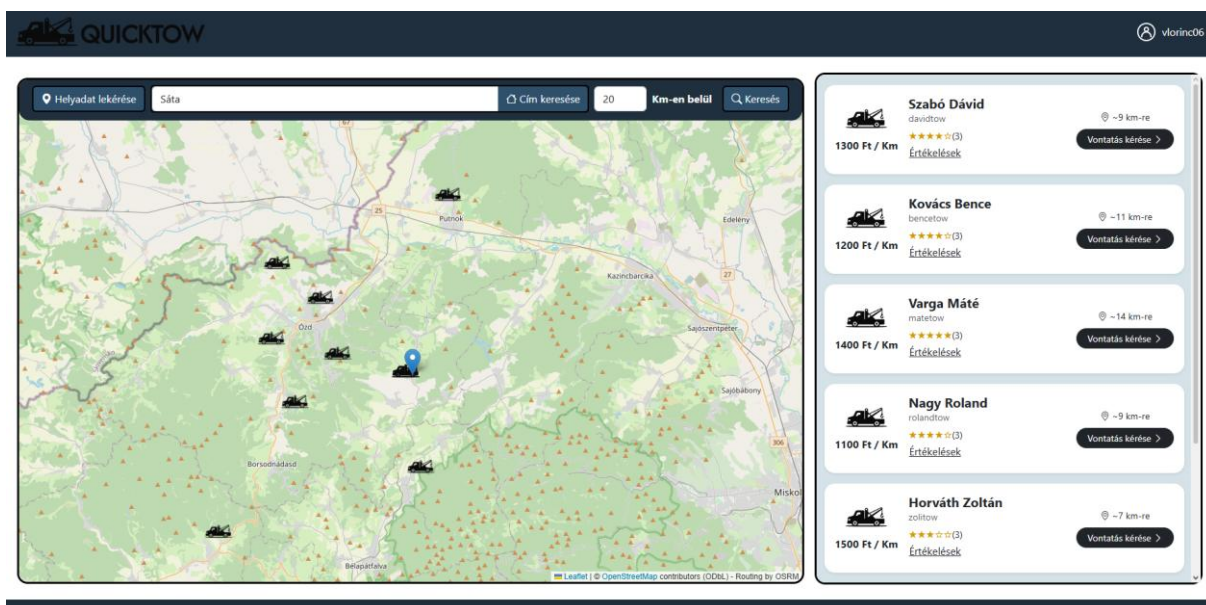
A regisztráció gombra kattintáskor felugró ablakban megjelenik két opció: Regisztrálás felhasználóként vagy regisztrálás autómentőként. A kettő hasonló regisztrációs felülethez vezet, viszont az autómentőnek több adatot kell megadni. Bejelentkezésnél csak felhasználónevet és jelszót kell megadni.

Bejelentkezés után használható lesz a térkép. Normál felhasználó számára a térkép tetején látható lesz egy keresősáv, amit az autómentők keresésére szükséges információk megadásához lehet használni.



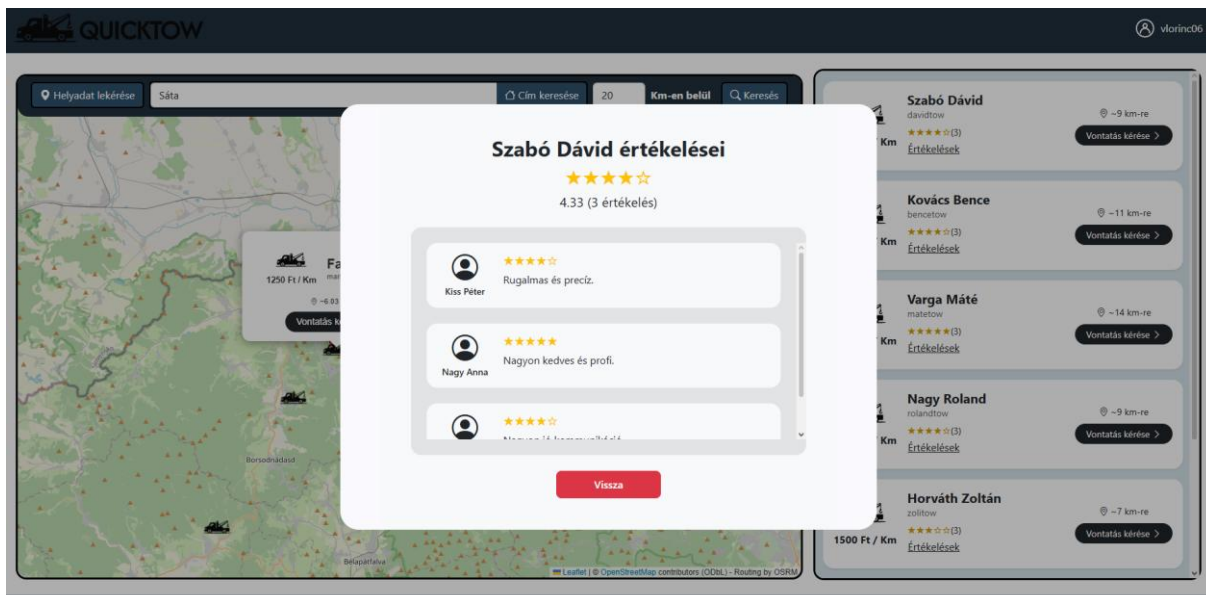
1. Főoldal normál felhasználó számára bejelentkezés után

Keresés után az elérhető autómentők megjelennek a térképen, illetve a térkép mellett lévő (kis képernyőkön a térkép alatt) listán.



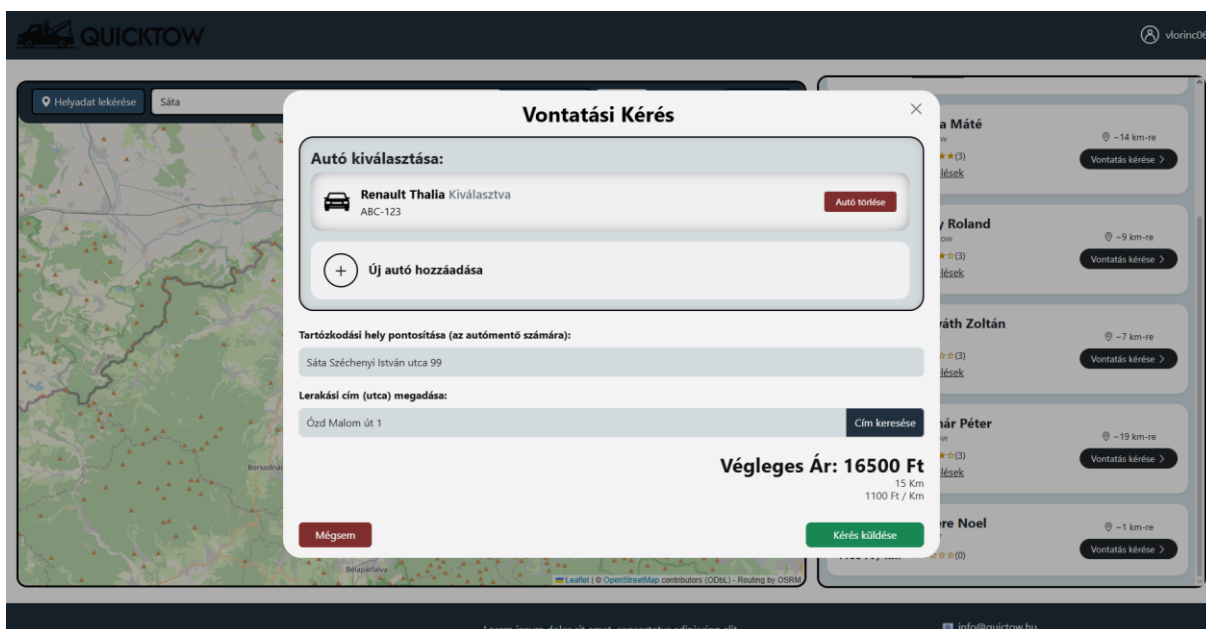
2. Főoldal autómentők keresése után

Az autómentők értékeléseit meg lehet nézni az „Értékelések” szövegre kattintva a nevük alatt. Ebből a felugró ablakot az ablakból kikattintással vagy a vissza gombra kattintással be lehet zárni.



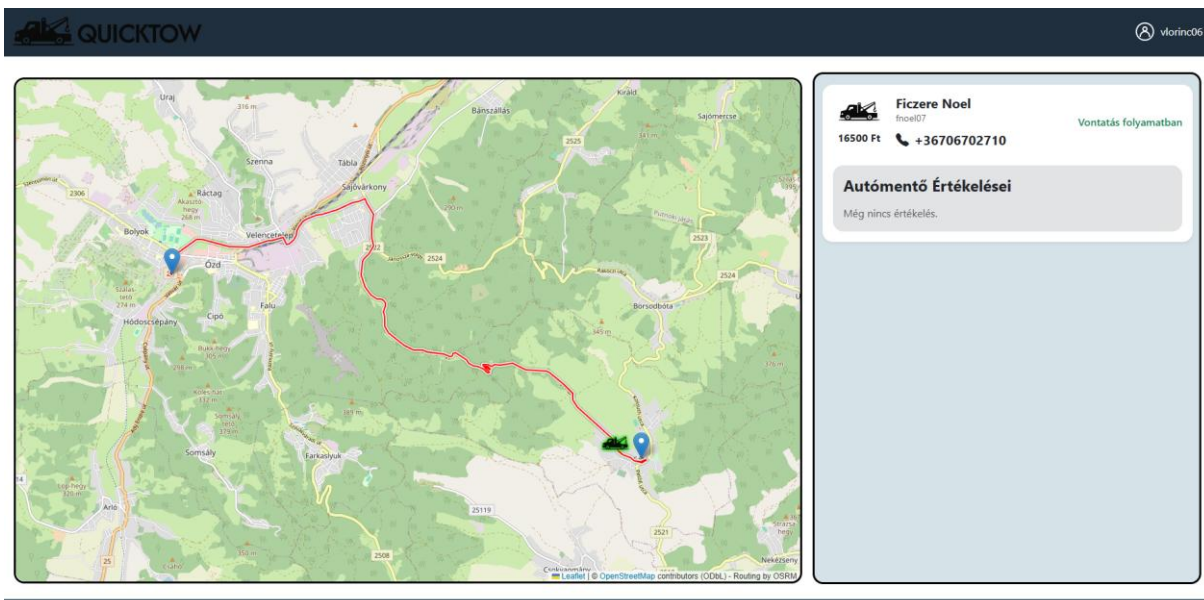
3. Értékelések felugró ablak

A vontatás kérése gombra kattintva (a listában vagy a térképen) a vontatási kérés ablak jelenik meg. Ebben a felhasználó ki tudja választani az autót, vagy új autót tud hozzáadni a fiókhoz. Ez alatt a címet kell pontosítani az autómentő számára, majd a lerakási pontot kell megadni. A lerakási pont megadása után a cím keresése gombra kattintva kiszámolja az alkalmazás hogy mennyi kilométer lesz a vontatás, illetve hogy mennyit kell érte fizetni. A kérés küldése gombra kattintva elküldi a kérést az autómentőnek és bezáródik a felugró ablak.



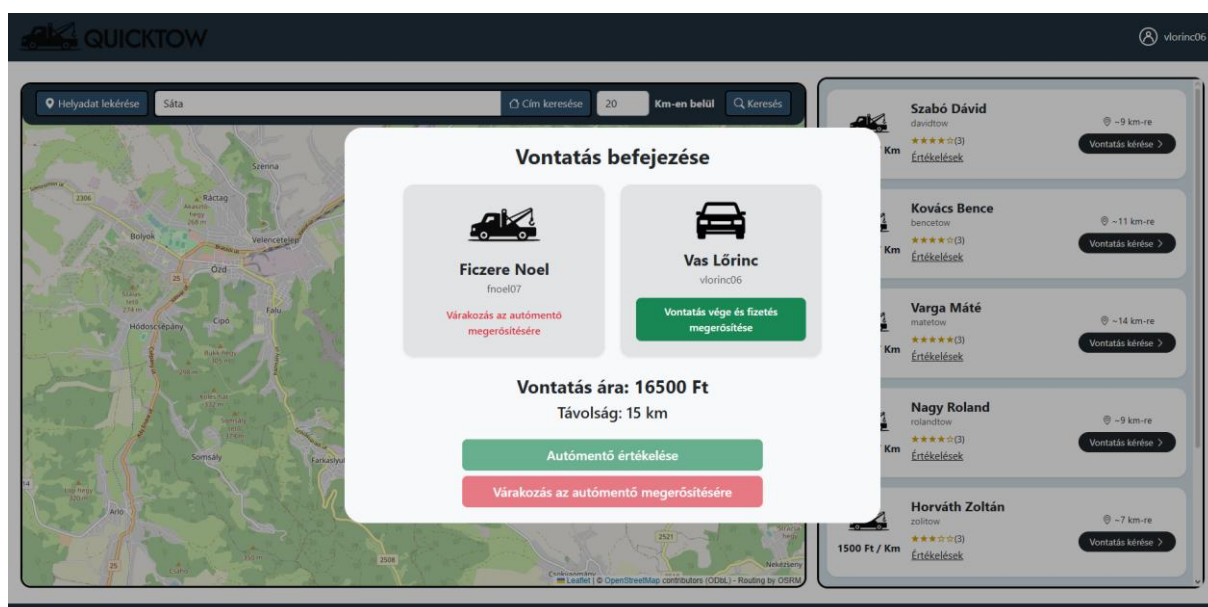
4. Vontatási kérés ablak

A kérés elküldése után, ha az autómentő elfogadja a kérést, erről visszajelzést kap a felhasználó is, illetve az útvonal megjelenik a térképen



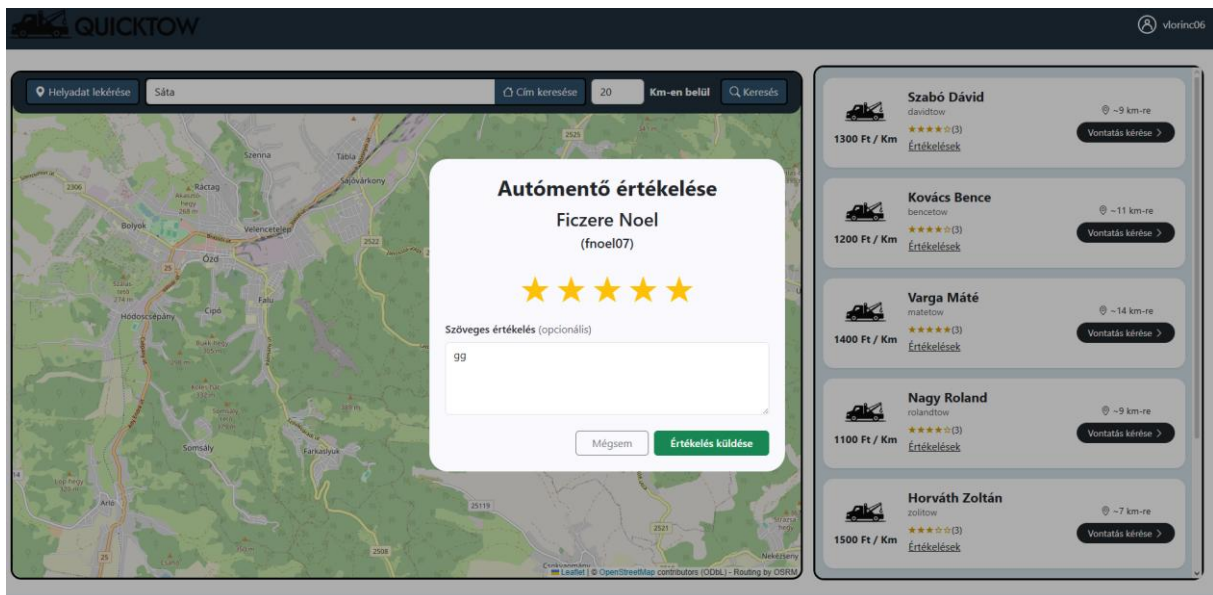
5. A kérés elfogadása után

A vontatás befejezése után felugró ablakban az autómentőnek és a felhasználónak is meg kell erősíteni a vontatás végét, csak utána lehet bezárni az ablakot.



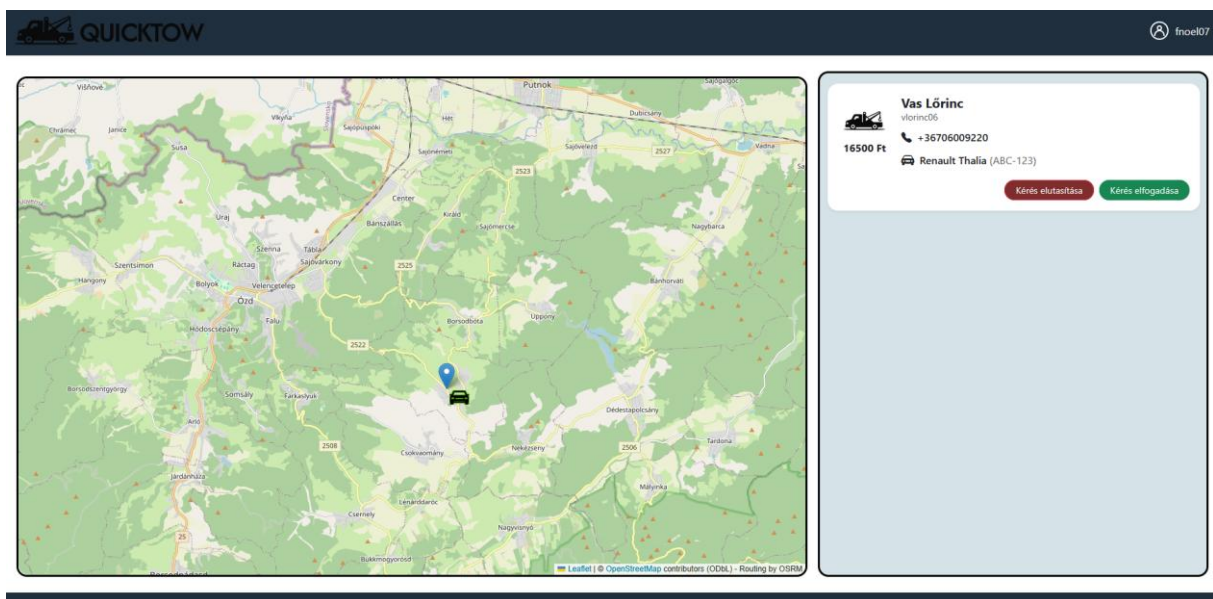
6. Vontatás befejezésének megerősítése

A megerősítés után a felhasználónak lehetősége van értékelést hagyni az autómentőre 1-5 csillagig, illetve szöveges értékelést is lehet hagyni.



7. Autómentő értékelése

Autómentő felhasználó esetén a főoldal kinézete megegyezik, viszont a térkép tetején nincs keresősáv. A bejövő vontatási kérések megjelennek a listán, illetve a térképen is. Itt az autómentő el tudja fogadni vagy el tudja utasítani a kérést.



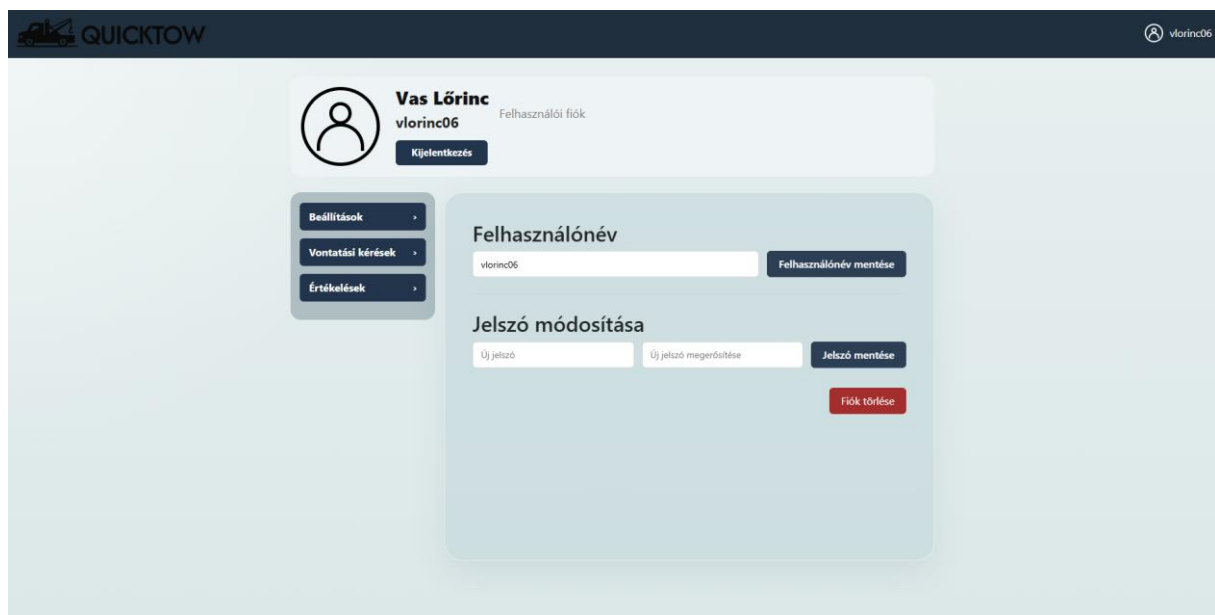
8. Bejövő vontatási kérés

Vontatás befejezésekor ugyan úgy meg kell erősíteni mind a kettő félnek, viszont az autómentő felhasználónak nem jelenik meg értékelés gomb.

iii) Profilbeállítások oldal leírása

A főoldalon a felhasználónévre kattintva a profilbeállítások oldal nyílik meg. Itt ki lehet jelentkezni, illetve a felhasználói típustól függően különböző menüpontok között lehet

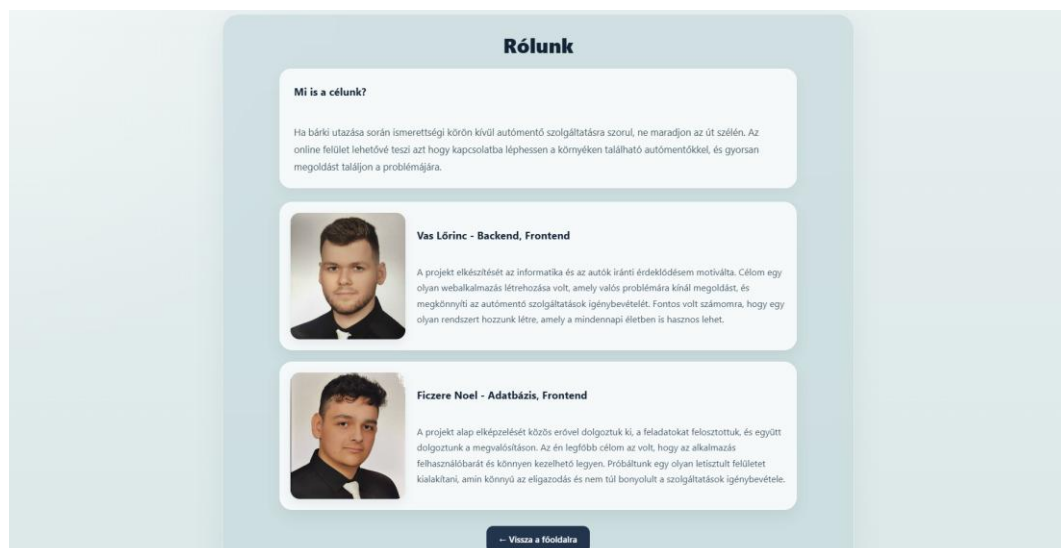
választani. Itt lehetséges a felhasználónév és jelszó (autómentő felhasználó esetén a kilométerár is) módosítása, előző vontatási kérések, illetve az értékelések megtekintése



9. Profilbeállítások oldal

iv) Rólunk oldal leírása

A rólunk oldalnak nincs külön funkcionálitása, a webalkalmazásról és készítőiről található rövid leírás.



10. Rólunk oldal

3. Fejlesztői dokumentáció

a) Alkalmazás fejlesztői és csoportmunka eszközök

Git(2.44)/GitHub: projekt tárolása, verziókezelés/menedzselés.

Discord: munka közbeni kommunikáció, segítségnyújtás.

b) Adatbázis

Visual Studio Code (1.87): Laravel, SQL kódok kezelése, adatbázis lekérések lekezelése.

XAMPP(8.2.12): Adatbázis felépítésének tesztelése, Apache szerver futtatása.

MySQL(8.4.x): Adatbázis tárolása.

c) Frontend

Visual Studio Code (1.87): Weboldal felépítésének megírása, weboldal tesztelése.

Angular(17.x): Weboldal keretrendszere.

Figma: Weboldal megtervezése, ötletelés.

d) Backend

Visual Studio Code (1.87): kód megírása, és tesztelés, Laravel API.

XAMPP(8.2.12): Adatbázis lokális futtatása, kezelése.

Laravel(11.x): Keretrendszer az API-hoz.

4. Adatbázis

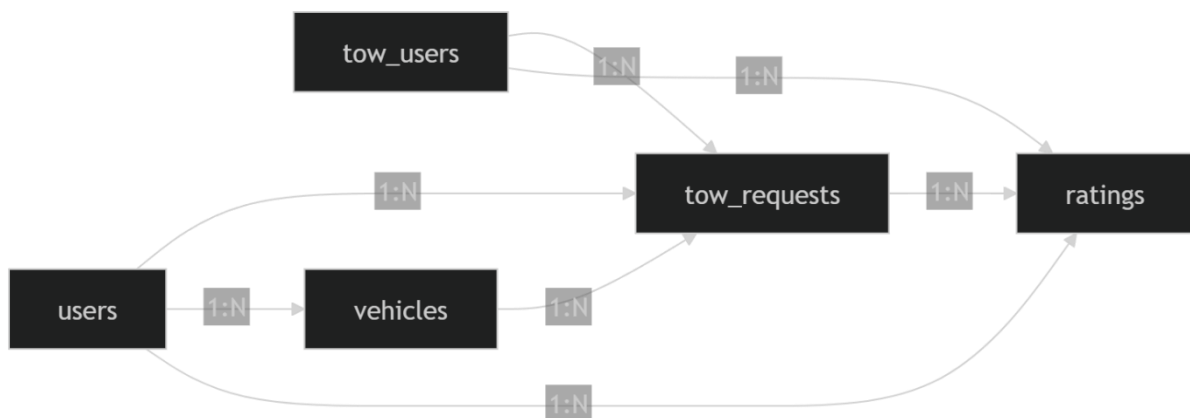
A QuickTow webalkalmazás adatbázisa az autómentő vállalkozók működését támogatja.

Az adatbázis fő célja, hogy összekapcsolja a felhasználókat, az autómentőket, a vontatási eseményeket, a járműveket, valamint a fizetéseket és az értékeléseket. Az adatbázis relációs szerkezetű, MySQL / MariaDB rendszerben készült.

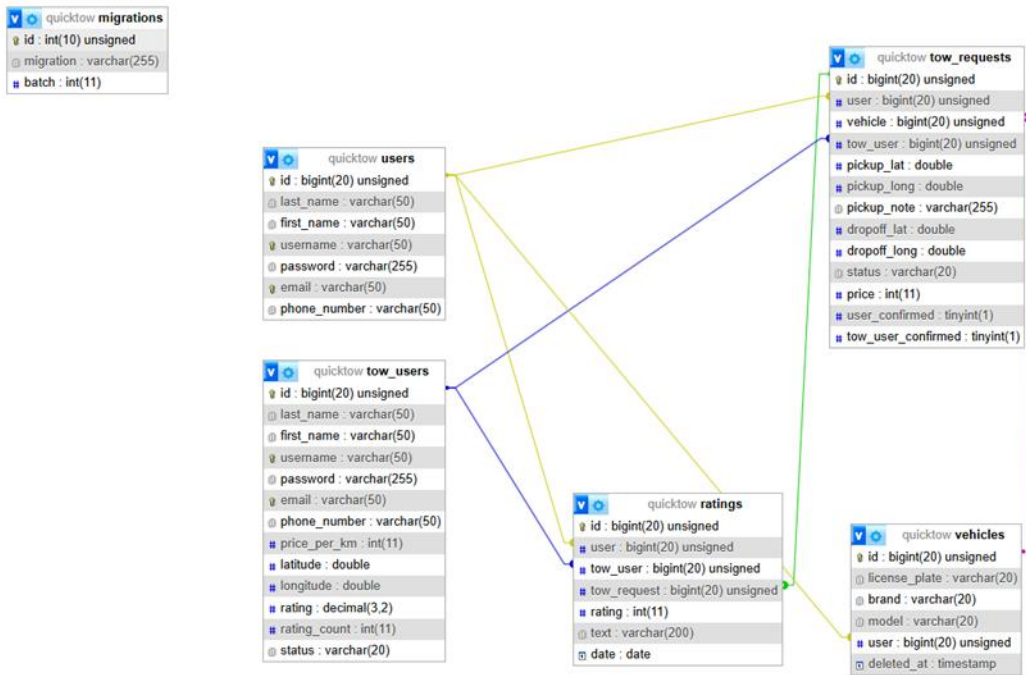
A főbb táblák:

- users
- tow_users
- vehicles
- tow_requests
- ratings

A táblák között idegen kulcsok teremtik meg a kapcsolatot. A táblázatban részletesen minden kapcsolat, és oszlop header el van magyarázva, alatta pedig egy ERM rajz található.



11. Adattáblák kapcsolatai



12. Adattáblák és mezők kapcsolatai

Adattáblák és mezők:

Vehicles adattábla

license_plate	A jármű rendszáma
brand	A jármű márkája
model	Az autó típusa
id	Az autó elérésére használt ID kapcsolatok: jarmu – vontatasikeres
user	Az autóval összezsátolt felhasználó ID-je
deleted_at	Törlés időpontja

Users adattábla

last_name	A felhasználó vezetékneve
first_name	A felhasználó keresztnéve
username	A bejelentkezéshez használt név
password	A bejelentkezéshez használt jelszó
email	A felhasználó e-mailje

phone_number	A felhasználó telefonszáma
id	A felhasználó eléréséhez használt ID kapcsolatok: • felhasználó – jármű • felhasználó – fizetés • felhasználó – értékeles • felhasználó vontatasikeres

TowUser adattábla

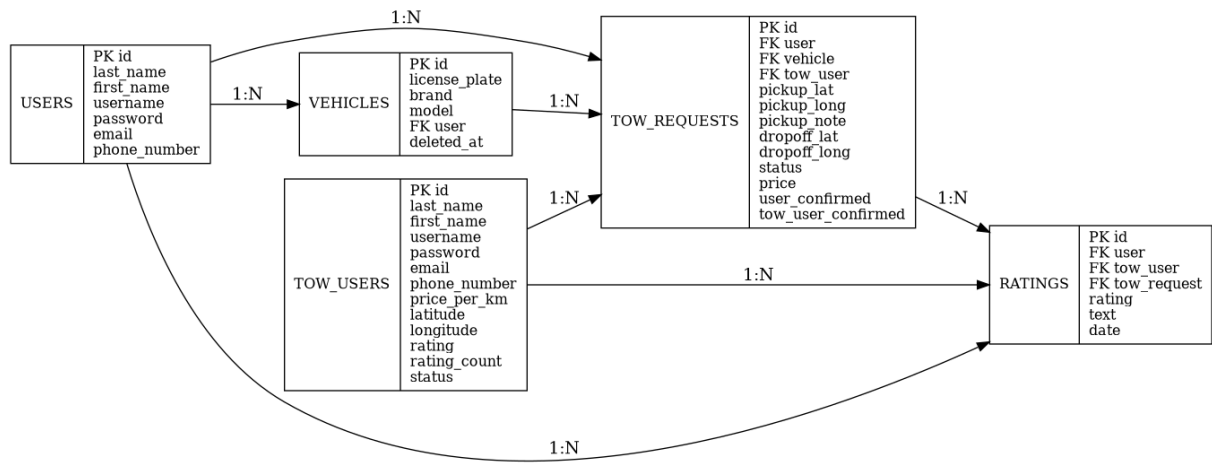
last_name	Az autómentő vezetékneve
first_name	Az autómentő keresztnéve
username	A bejelentkezéshez használt név
password	A bejelentkezéshez használt jelszó
email	Az autómentő email címe
phone_number	Az autómentő telefonszáma
latitude	Az autómentő koordinátájának X értéke
longtitude	Az autómentő koordinátájának Y értéke
status	Az autómentő jelenlegi státusza
rating	Az autómentő értékelése
rating_count	Az autómentő értékeléseinek száma
price_per_km	Az autómentő árazása
id	Az autómentő eléréséhez használt ID kapcsolatok: • automento – fizetes • automento – értékeles • automento – vontatasikeres

Ratings adattábla

id	Az értékelés eléréséhez használt ID kapcsolatok: nincs, a weboldalon elérhető.
user	Az értékelést író felhasználó ID-je
tow_user	A vontatást elvégző ID-je
tow_request	A vontatás ID-je
rating	Az értékelés számban
text	Az értékelésnél hagyott leírás
date	Az értékelés időpontja

TowRequests adattábla

id	A vontatás eléréséhez használt ID kapcsolatok: vontatasikeres - ertekeles
user	A vontatásban résztvevő felhaszn. ID-je
vehicle	A vontatott jármű ID-je
tow_user	A vontató ID-je
pickup_lat	A felvételpont koordináta X értéke
pickup_long	A felvételpont koordináta Y értéke
pickup_note	A felhasználó által kért pontosított felvételi pont
dropoff_lat	A lerakási pont koordináta X értéke
dropoff_long	A lerakási pont koordináta Y értéke
statusz	A vontatás státusza
ar	A vontatás ára
user_confirmed	A vontatást kérő személy szeretné lezárni a vontatást
tow_user_confirmed	A vontatást végrehajtó személy szeretné lezárni a vontatást



13. Adattáblák kapcsolatai és mezői

5. Frontend

A webalkalmazás frontend része Angular keretrendszerben készült HTML, Bootstrap, CSS és TypeScript használatával. Feladata a felhasználói felület megjelenítése, valamint a backend API-val való kommunikáció biztosítása.

A frontendben az adatok kezelésére TypeScript interface-ek szolgálnak. Ezek határozzák meg, hogy az alkalmazás milyen szerkezetű adatokat vár az API-tól, illetve milyen formában küldi vissza azokat. Külön interface tartozik a felhasználókhoz, autómentőkhöz, járművekhez, vontatási kérelmekhez és az értékelésekhez. Ahol a backend több objektumot ágyaz egymásba (azaz lekérésnél a külső kulcshoz tartozó adatokat is visszaadja), külön interface van a GET és a POST / PUT kérésekhez.

```
Frontend > src > app > models > rating.models > rating
1  import { TowUser } from "../towuser.model"
2  import { User } from "../user.model"
3
4  export interface RatingPost{
5      user: number,
6      tow_user: number,
7      tow_request: number,
8      rating: number,
9      text?: string,
10     date?: string
11 }
12
13 export interface Rating{
14     id: number,
15     user: User,
16     tow_user: TowUser,
17     tow_request: number,
18     rating: number,
19     text?: string,
20     date?: string
21 }
```

14. A Rating interface-ek

A service-ek legfőbb feladata a frontend és backend közötti kommunikáció, viszont ezen kívül külső (third party) API-kat is kezelnek, illetve az alkalmazás állapotának kezelését is megvalósítják.

A UserService legfőbb feladata a felhasználó adatainak lekérése, feltöltése, vagy módosítása. Az Angularban a HttpClient segítségével lehet a backendnek kéréseket küldeni, ezt minden

service-ben be kell injektálni használat előtt. Külön metódusok vannak a felhasználók lekérésére, hozzáadására, frissítésére és törlésére. Ezekben majd a komponensekben kell feliratkozni. Külön változóban el van tárolva a backend API címének az alapja, így ha változna nem kell mindenhol átírni ahol használva van. A UserService emellett tárolja a felhasználó adatait egy signal segítségével, valamint opcionálisan a felhasználó földrajzi helyzetét koordináták formájában. Ez lehetővé teszi, hogy az alkalmazás különböző részei hozzáférjenek ezekhez az adatokhoz.

A TowUserService az autómentő felhasználók adatainak kezelésére szolgál. Működése hasonló a UserService-hez, azonban kiegészül speciális, az autómentők működéséhez szükséges funkciókkal. A service lehetőséget biztosít az autómentők lekérdezésére földrajzi koordináták alapján. A getTowUsers metódus egy adott pozícióhoz és sugárhoz tartozó autómentőket adja vissza, ami lehetővé teszi a felhasználó számára a közelben lévő szolgáltatók megjelenítését. A service egy selectedTowUser signal segítségével tárolja a kiválasztott autómentő azonosítóját, ami az alkalmazás különböző részei között is elérhetővé teszi ezt az információt.

```
getTowUsers(lat: number, lng: number, radius: number): Observable<any> {  
  const apiUrl = `http://127.0.0.1:8000/api/towusers/radius/?lat=${lat}&long=${lng}&radius=${radius}`  
  return this.http.get<TowUser[]>(apiUrl);  
}
```

15. A getTowUsers metódus a TowUserService-ben

A VehicleService a felhasználó járműinek adatainak a kezelésére szolgál. Egy selected signal itt is megtalálható, ami a kiválasztott autót tárolja.

A TowRequestService a vontatási kéréseket kezeli. Több módon lehet lekérni ezeket: Id alapján (egy vontatási kérést ad vissza), autómentő id alapján (minden autómentőhöz tartozó vontatás), illetve felhasználó id alapján (minden felhasználóhoz tartozó vontatás.)

```

getTowRequestById(id: number){
  const apiUrl = `http://127.0.0.1:8000/api/towrequests/id/${id}`

  return this.http.get<TowRequest>(apiUrl);
}

getTowRequestsByTowUser(id: number){
  const apiUrl = `http://127.0.0.1:8000/api/towrequests/getby/tow_user/${id}`

  return this.http.get<TowRequest[]>(apiUrl);
}

getTowRequestsByUser(id: number){
  const apiUrl = `http://127.0.0.1:8000/api/towrequests/getby/user/${id}`

  return this.http.get<TowRequest[]>(apiUrl);
}

```

16. Vontatások lekérésének típusai a TowRequestService-ben

A RatingService segítségével szintúgy le lehet kérni az értékeléseket felhasználó és autómentő id alapján. Új értékelés hozzáadása illetve értékelés törlése metódusok itt is megtalálhatóak.

Az AuthService feladata hogy tárolja hogy van-e bejelentkezve felhasználó vagy autómentő, és ha van, milyen adatokkal. Ezt a currentUser signalban tárolja, és ebből további értékeket is számol ki, például hogy milyen típusú felhasználó van bejelentkezve. Bejelentkezésnél ezek a signalok értéket kapnak, illetve az adatokat lementi localStorage-be is.

```

currentUser = signal<AuthUser | null>(null);

isLoggedIn = computed(() => this.currentUser() !== null);
isTowUser = computed(() => this.currentUser()?.type === 'towUser');
isRegularUser = computed(() => this.currentUser()?.type === 'user');

isRequesting = signal<boolean>(false);

```

17. Signalok az AuthService-ben

A LeafletMapService a térkép működéséhez szükséges third party API-kat tartalmaz. A getAddress két koordinátából címet (pl. Város + utca) tud visszaadni a Nominatim segítségével. A searchStreet ennek az ellenkezőjét csinálja, paraméterben egy címet kap és ezt alakítja át koordinátákra. Ezek mellett a service-ben található a térképen megjelenő jelölők kattintásakor felugró ablakok HTML kódja.

```

    getAddress(lat: number, lng: number): Observable<any> {
      const apiUrl = `https://nominatim.openstreetmap.org/reverse?lat=${lat}&lon=${lng}&format=json`;
      return this.http.get<any>(apiUrl);
    }
  }

```

18. A getAddress metódus a LeafletMapService-ben

A LocationTrackingService feladata a bejelentkezett autómentők koordinátáinak lekérése 30 másodpercenként, és ennek feltöltése az adatbázisba, azaz ez a service csak akkor van használva, ha autómentő van bejelentkezve (ha az AuthService-ben a currentUser típusa 'towUser'). Az updateTowUserLocation metódus a navigator.geolocation.getCurrentPosition használatával lekéri a böngészőn keresztül a tartózkodási helyet. Ha ez nem elérhető, akkor automatikusan egy alap értékre tér vissza. Ez a metódus a startTowUserLocationUpdates-ben intervallban van lefuttatva 30 másodpercenként. A stopTowUserLocationUpdates leállítja az intervallt, azaz nem lesz többször frissítve az autómentő pozíciója.

```

navigator.geolocation.getCurrentPosition(
  (position) => {
    const lat = position.coords.latitude;
    const lng = position.coords.longitude;

    this.currentLocation.set({ lat, lng });

    console.log('Tow user location:', lat, lng)

    this.towUserService.updateTowUser(towUserId, { latitude: lat, longitude: lng } as TowUser).subscribe({
      next: (res) => {
        console.log('Location updated', res);
      },
      error: (err) => {
        console.error('location update failed', err)
      }
    });
  },
  {
    enableHighAccuracy: true,
    timeout: 5000,
    maximumAge: 0
  }
);

```

19. A helyadatok lekérése a LocationTrackingService-ben

A felhasználói felület Angular komponensekből épül fel. Minden komponens egy adott nézethez vagy funkcióhoz kapcsolódik, és általában három fő részből áll: egy HTML, egy CSS és egy TypeScript fájlból.

A Navbar komponensben található az oldalak tetején található navigációs sáv. Ha a felhasználó nincs bejelentkezve, akkor a bejelentkezés és a regisztráció gomb jelenik meg rajta, de viszont, ha bejelentkezett akkor megjelenik a neve és egy kis profil ikon. Az oldalak közötti közlekedés elősegítésére szolgál, például a logó megnyomásával vissza tudunk térni a főoldalra.

A footer komponens minden oldalon megtalálható, a képernyő alján elhelyezkedő sávot biztosítja. Megtalálható a projekt fő célja, elérési lehetőségek, és a rólunk oldal.

A TowMapPage a webalkalmazás fő része. Önmagában nincs funkcionálitása, viszont két másik komponenst jelenít meg: a LeafletMap és a TowUserList komponenst, ezektől fogad, illetve ad át adatot (Input és Output segítségével). Ezeknek teljes működésük csak bejelentkezés után lesz elérhető.

A RegistryComp feladata az, hogy egy gomb segítségével a felhasználó tudjon egy fiókot regisztrálni. Lehetőség van mind a kettő típusú fiókot regisztrálni, amit a felhasználó gomb megnyomásával dönt el.

```
<div *ngif="selectedType === 'user'">
  
  <button class="registry-back-btn" type="button" (click)="back()">← Vissza</button>

  <h2>Regisztráció</h2>

  <form (ngSubmit)="register()" class="registry-form">
    <div class="registry-row">
      <input type="text" placeholder="Vezetéknév" name="last_name" [(ngModel)]="last_name" required>
      <input type="text" placeholder="Keresztnév" name="first_name" [(ngModel)]="first_name" required>
    </div>

    <input type="text" placeholder="Felhasználónév" name="username" [(ngModel)]="username" required>
    <input type="password" placeholder="Jelszó" name="password" [(ngModel)]="password" required>
    <input type="email" placeholder="Email" name="email" [(ngModel)]="email" required>
    <input type="text" placeholder="Telefonszám" name="phone_number" [(ngModel)]="phone_number" required>

    <button type="submit" class="primary-btn">
      Regisztrálás felhasználóként
    </button>
  </form>
</div>
```

20.RegistryKomp HTML

a selectedType változó két értéket kaphat (user/tow), és ez alapján dönti el hogy melyik regisztrációs űrlapod dobja fel a felhasználónak. Miután a felhasználó megnomja a regisztráció gombot, az adatok amit megadott elmentődnek a payload változóba, és utána egy kérdésben elküldi egy post metódusként az adatbázisnak. Ha valami miatt például a validátor visszautasítja a kérést, akkor azt a hibát a frontend lekezeli, és a felhasználó felé továbbítja. Hibakeresés végett a konzolra kiírásra kerül a regisztráció típusa, a kérés elérési útja, és a payload változó összes értéke.

```

const payload: any = {
  last_name: this.last_name,
  first_name: this.first_name,
  username: this.username,
  password: this.password,
  email: this.email,
  phone_number: this.phone_number,
};

if (this.selectedType === 'tow') {
  payload.price_per_km = this.priceperkm;
}

const url =
  this.selectedType === 'tow'
    ? `${this.apiBase}/towusers`
    : `${this.apiBase}/users`;

console.log('REGISTER type:', this.selectedType);
console.log('REGISTER url:', url);
console.log('REGISTER payload:', payload);

this.http.post(url, payload).subscribe({
  next: (res: any) => {
    console.log('REGISTER OK:', res);
    alert('Sikeres regisztráció!');
    this.closeModel();
  },
  error: (err) => {
    console.log('REGISTER ERROR:', err);
    alert(`Hiba regisztráció közben: ${err?.status ?? ''}`);
  },
});

```

21. Register metódus

A Login komponens a regisztrációhoz hasonlóan működik, csak teljesen más backend kódot használ. Ha a felhasználó be szeretne jelentkezni meg kell adnia a fiókja felhasználónevét és jelszavát. A bejelentkezés gomb felett egy kipipálható mező található, ezzel jelezzük a rendszer felé azt hogy mi milyen típusú fiókba szeretnénk bejelentkezni.

```

login() {
  this.auth.login(this.username, this.password, this.towuser).subscribe({
    next: (res: any) => {
      const user = res.user;

      this.loggedUser = user;

      const authUser: AuthUser = {
        id: user.id,
        type: this.towuser ? 'towUser' : 'user',
        username: user.username,
        first_name: user.first_name,
        last_name: user.last_name,
        email: user.email,
        phone_number: user.phone_number,
      };

      this.authService.setUser(authUser);

      if (authUser.type == "towUser") {
        this.towUserService.updateTowUser(authUser.id, {
          status: "available"
        }).subscribe()
      }

      alert('Sikeres bejelentkezés');
      this.closeModel();
    },
    error: (err) => {
      if (err.status === 404) {
        alert('Nem található ilyen felhasználó.');
      }
    }
  });
}

```

22. Login metódus

a login metódus az auth. login segítségével eldönti, hogy melyik végpontra küldjön kérést az adatokkal, hiba esetén lekezelés után hibát küld a felhasználónak. Ha minden sikeresen lefutott, a szerver elküldi annak a fióknak az összes adatát, ami megegyezik a felhasználónévvel és a jelszóval. Ha a felhasználó vontatós, automatikusan bejelentkezés után megváltoztatja az elérhetőségét elérhetőre.

A LeafletMap komponens a webalkalmazás egyik központi eleme, amely a térképes felület megjelenítéséért és az ahhoz kapcsolódó interakciók kezeléséért felelős. A komponens Leaflet alapú térképet jelenít meg, kezeli a felhasználó helyzetét, az autómentők megjelenítését, valamint a vontatási kérelmek állapotához kapcsolódó térképes műveleteket.

A komponens működése függ a bejelentkezett felhasználó típusától. Bejelentkezés nélkül a térkép elhomályosítva jelenik meg, és csak a bejelentkezési vagy regisztrációs lehetőség érhető el. Normál felhasználó esetén lehetőség van helyadat lekérésére, cím keresésére és a megadott keresési paraméterek alapján a közeli autómentők keresésére. Autómentő felhasználó esetén a komponens az aktuális vontatási kérelmeket jeleníti meg a térképen.

A komponens létrehozza a Leaflet térképet, OpenStreetMap csempéket használ, és tartalmaz tartalék térképreteget is hiba esetére.

```
const tiles = L.tileLayer(  
  'https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png',  
  {  
    maxZoom: 18,  
    minZoom: 8,  
    attribution:  
      '&copy; <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a> contributors (ODbL) - Routing by OSRM',  
  }  
);  
  
const fallbackTiles = L.tileLayer(  
  'https://{s}.basemaps.cartocdn.com/light_all/{z}/{x}/{y}{r}.png',  
  {  
    maxZoom: 18,  
    minZoom: 8,  
    attribution:  
      '&copy; OpenStreetMap contributors &copy; CARTO',  
  }  
);
```

23. OpenStreetMap és hiba esetén CARTO térkép csempék

A getLocation függvény a felhasználó helyadatainak a lekérésére szolgál. Ha engedélyezve van a helyadatokhoz való hozzáférés és sikeresen megtörténik a koordináták lekérése, akkor a felhasználó pozícióját a térképen jelölni fogja, majd ez alapján fog a közelben lévő autómentők keresése is történni. Ha a helyadatok lekérése nem elérhető, vagy a felhasználó nem szeretné használni, cím alapján is lehet keresni a searchStreet függvénnyel. Ez a LeafletMapService-ben lévő metódust használja, ennek a visszatérési értéke alapján állítja be a térképen a tartózkodási helyet.


```

this.mapService.searchStreet(this.street).subscribe({
  next: (res) => {
    if (res.length < 1) {
      console.log('Nem találtuk a keresett címet');
      return;
    }

    const lat = parseFloat(res[0].lat);
    const lng = parseFloat(res[0].lon);

    this.map.setView([lat, lng], 15);

    if (this.userMarker) {
      this.map.removeLayer(this.userMarker);
    }

    if (this.userCircle) {
      this.map.removeLayer(this.userCircle);
    }

    this.userMarker = L.marker([lat, lng])
      .addTo(this.map)
      .bindPopup(res[0].display_name)
      .openPopup();
  },
});

```

24. Cím keresése.

Ha van a felhasználónak a térképen megjelölt pozíciója és van megadva keresési sugár (tehát maximum hány km-re lehetnek az autómentők), a keresés gomb megnyomásával a search függvény fut le. Ennek feladata a megadott értékek alapján kérést küldeni a backendnek, ami visszaadja a megfelelő autómentőket. Az elérhető állapotú autómentőket beleteszi a towUsers listába (ezeket majd Output segítségével megkapja a TowMapPage komponens), és megjeleníti őket a térképen a createMarkerWithPopup függvény használatával.

```

this.towUserService.getTowUsers(lat, lng, this.radius).subscribe({
  next: (users: TowUser[]) => {
    const validTowUsers = users.filter(
      (u: TowUser): u is TowUser & { latitude: number; longitude: number } =>
        u.latitude !== undefined && u.longitude !== undefined
    );

    this.towUsers = validTowUsers.filter((u) => u.status === 'available');

    this.towMarkers.forEach((marker) => {
      this.map.removeLayer(marker);
    });
    this.towMarkers.clear();

    validTowUsers.forEach((towUser) => {
      const marker = this.createMarkerWithPopup(
        towUser.latitude,
        towUser.longitude,
        this.getTowIconByStatus(),
        () => this.mapService.popupHtml(towUser),
        () => this.userMarker?.getLatLng() ?? null,
        true,
        (popupElement) => {
          setTimeout(() => {
            const requestBtn =
              popupElement.querySelector('.tow-request-btn');

            requestBtn?.addEventListener('click', () => {
              if (towUser.id != null) {
                this.openDialog(towUser.id);
              }
            });
          }, 0);
        }
      );
    });
  }
});

```

25. Autómentők lekérése és megjelenítése a térképen

Az `ngOnInit`-tel a komponens betöltésekor ha felhasználó van bejelentkezve indul egy interval, ami 5 másodpercenként megnézni hogy van-e a felhasználónak olyan vontatási kérése ami elfogadásra vár, vagy folyamatban van. Ha talál ilyet, a `loadSelectedTowUserMarker` függvény segítségével a térképen csak azt az autómentőt fogja megjeleníteni, amelyikhez a kérés tartozik. Ha a kérés már folyamatban van, meghívja a `drawInProgressRoute` metódust is, ami az útvonaltervet megjeleníti a térképen. Ha autómentő van bejelentkezve, akkor az interval funkciója hasonló, a bejövő vontatási kéréseket, vagy a már folyamatban lévőket jeleníti meg a `loadTowRequestMarkers` függvénnyel.

```

if (userType === 'user') {
  interval(5000)
    .pipe(
      startWith(0),
      switchMap(() => this.towRequestService.getTowRequestsByUser(userId)),
      takeUntil(this.destroy$)
    )
    .subscribe({
      next: (requests) => {
        const activeRequest = requests.find(
          (r) =>
            r.status === 'awaiting response' || r.status === 'in progress'
        );

        if (!activeRequest) {
          if (this.towUserService.selectedTowUser() !== null) {
            this.towUserService.selectedTowUser.set(null);
            this.towMarkers.forEach((marker) => this.map.removeLayer(marker));
            this.towMarkers.clear();
          }
        } else if (activeRequest.tow_user?.id) {
          this.towUserService.selectedTowUser.set(activeRequest.tow_user.id);
          this.loadSelectedTowUserMarker(
            activeRequest.tow_user.id,
            activeRequest.status
          );
        }
      }
    });
}

```

26. Az interval egy része az ngOnInit-ben

```

loadSelectedTowUserMarker(towUserId: number, requestStatus?: string) {
  const userType = this.auth.currentUser()?.type;

  if (userType === 'user') {
    this.towUserService.getTowUserById(towUserId).subscribe({
      next: (towUser) => {
        if (towUser.latitude == null || towUser.longitude == null) {
          return;
        }

        this.towMarkers.forEach((marker) => this.map.removeLayer(marker));
        this.towMarkers.clear();

        const marker = L.marker([towUser.latitude, towUser.longitude], {
          icon: this.getTowIconByStatus(requestStatus),
        }).addTo(this.map);

        this.towMarkers.set(towUserId, marker);
      },
    });
  }
}

```

27. A loadSelectedTowUserMarker függvény

A térképen a jelölőkre kattintva a felugró ablakról (popup) indítható vontatási kérelem felhasználóknak, autómentők pedig tudnak elfogadni vagy elutasítani kérést ezekről.

```
acceptRequestFromMap(request: TowRequest) {  
  if (!request.id) return;  
  
  const updated = {  
    ...request,  
    status: 'in progress',  
  };  
  
  this.towRequestService.updateTowRequest(request.id, updated).subscribe(() => {  
    this.map.closePopup();  
  });  
}
```

28. Metódus a kérés elfogadására popup-ról

A TowUserList komponens a térkép mellett, vagy kisebb képernyőkön alatta helyezkedik el. Célja, hogy az elérhető autómentőket, vagy a vontatási kéréseket átláthatóbban, több részlettel mutassa.

A komponens működése a bejelentkezett felhasználó típusától függ. Normál felhasználó esetén a közelben található autómentők listája jelenik meg (a szülő komponensből kapja az adatokat Input segítségével), ahol megtekinthetők az alap adatok (név, ár, értékelés), valamint lehetőség van vontatási kérés indítására. Autómentő felhasználó esetén a beérkező vontatási kérelmek jelennek meg listában, ahol a felhasználó elfogadhatja vagy elutasíthatja a kéréseket, illetve folyamatban lévő vontatás esetén annak állapotát követheti.

Ha a felhasználó rákattint a vontatás kérése gombra, akkor a TowRequestWindow komponenst nyitja meg felugró ablakban (Angular dialog). A vontatás kérésének további részei ebben bonyolódnak le. E mellett az autómentők értékeléseit is meg lehet tekinteni az Értékelések szövegre kattintva. Ez a ViewRatingsWindow komponenst nyitja meg.

```

openDialog(id: number) {
  this.towUser.selectedTowUser.set(id)
  const dialogRef = this.dialog.open(TowRequestWindow, {
    width: '1000px',
    maxWidth: '95vw',
    maxHeight: '90vh',
    disableClose: true
  })

  dialogRef.afterClosed().subscribe(() => {
    this.checkRequests();
  })
}

```

29. Dialog megnyitására szolgáló függvény

Ha normál felhasználó van bejelentkezve, akkor 5 másodpercenként lefut a checkRequests metódus, ami lekéri az adott felhasználóhoz tartozó elfogadásra váró, folyamatban lévő, vagy elutasított vontatási kérést, és azt jeleníti meg a listában. Ha van ilyen kérés, addig a felhasználó nem tud újat indítani, egyszerre csak egy aktív vontatási kérés tartozhat egy felhasználóhoz.

```

checkRequests() {
  const userId = this.auth.currentUser()?.id;
  const userType = this.auth.currentUser()?.type;
  if (!userId) return;

  if (userType === "user") {
    this.towRequest.getTowRequestsByUser(userId).subscribe({
      next: (res: TowRequest[]) => {
        const active = res.find(r =>
          r.status === 'awaiting response' || r.status === 'in progress'
        );

        const denied = res.find(r => r.status === 'denied');

        const confirming = res.find(r => r.status === 'confirming')

        if (confirming && !this.userConfirming) {
          this.userConfirming = true;
          this.openEndDialog(confirming);
        }

        if (!confirming) {
          this.userConfirming = false;
        }

        if (active?.tow_user.id) {
          this.loadTowUserRatings(active.tow_user.id)
        }

        this.towRequests.set(active ? [active] : []);
        this.auth.isRequesting.set(!active);
        this.requestDenied.set(!active && !!denied);
        this.deniedRequest.set(denied ?? null)
      }
    });
  }
}

```

30. A checkRequests függvény

Ha autómentő felhasználó van bejelentkezve, akkor a startPollingRequests függvény fut le egyszer, és ebben 5 másodpercenként lekéri az adatbázisból hogy van-e az adott autómentő id-jéhez tartozó elfogadásra váró vagy aktív vontatási kérés. Ha nincs aktív vontatás, akkor az összes bejövő kérelem megjelenik. Ezeknél két opció van: Kérés elfogadása és kérés elutasítása. Ha van aktív vontatás, akkor csak azt fogja látni az autómentő. A vontatás befejezése gombra kattintva a TowRequestEndWindow komponenst nyitja meg.

A ViewRatingsWindow komponens egy felugró ablakként (dialog) megjelenő komponens, amely egy adott autómentőhöz tartozó értékeléseket jeleníti meg. A komponens a megnyitásakor megkapja a kiválasztott autómentő azonosítóját, majd a RatingService segítségével lekéri az adott autómentőhöz tartozó értékeléseket. A kapott adatokat egy listában jeleníti meg, ahol látható az értékelést író felhasználó neve, az értékelés szövege, valamint a csillagokkal jelölt pontszám.

A komponens egy számított értéket (computed) is használ az átlagos értékelés megjelenítésére, amely csillagok formájában jelenik meg a felületen.

```
averageRatingStars = computed(()=>{
  const ratings = this.ratings();
  if(ratings.length === 0) return '☆☆☆☆'

  const rating = ratings[0].tow_user.rating;
  const rounded = Math.round(rating);

  return '★'.repeat(rounded) + '☆'.repeat(5-rounded)
})
```

31. Értékelés alapján csillagok kiszámolása

A TowRequestWindow komponens egy felugró ablakban (dialog) megjelenő komponens, amely a vontatási kérés létrehozásáért felelős. Megnyitásakor paraméterben megkapja a kiválasztott autómentő id-jét, mivel ez a kérés küldéséhez szükséges. A komponens lehetőséget biztosít a felhasználó számára, hogy hozzáadja vagy kiválassza a vontatandó járművet, megadja a felvételi hely pontosítását, valamint a lerakási címet. Új jármű hozzáadásánál feltölti a járművet az adatbázisba a VehicleService segítségével.

```
saveCar() {
  if (!this.car.brand || !this.car.model || !this.car.license_plate) {
    return;
  }

  this.vehicles.addVehicle(this.car).subscribe({
    next: (vehicle) => {
      console.log("Jármű hozzáadva:", vehicle)

      this.vehicles.getVehicles(this.auth.currentUser().id);
    }
  })
}
```

32. A saveCar függvény, feltölti az adatbázisba az új autót

A lerakási cím megadásakor a komponens a LeafletMapService segítségével koordinátákká alakítja a megadott címet, majd kiszámolja az útvonal hosszát és a végleges árat az autómentő kilométer alapú díja alapján.

```

this.map.searchStreet(this.dropoffAddress).subscribe({
  next: (res: any[]) => {
    if(res.length === 0)
    {
      this.addressMessage.set("A keresett cím nem található")
      return
    }
    this.addressMessage.set("");
    this.setDropoff(parseFloat(res[0].lat), parseFloat(res[0].lon));
    this.calculateRoute();
  },
});

```

33. Megadott cím átalakítása a searchStreet függvényben

A komponens signalokban kezeli a vontatási kéréshez szükséges adatokat, például a kiválasztott autómentőt, a távolságot, vagy a kiszámolt árat, így a felület automatikus frissül.

A „Kérés küldése” gomb megnyomásakor a komponens létrehoz egy vontatási kérés objektumot, amely tartalmazza a szükséges adatokat (felhasználó, jármű, koordináták, ár), majd ezt elküldi a backendnek a TowRequestService segítségével.

```

sendRequest() {
  const selectedTowUser = this.selectedTowUser()
  const fullPrice = this.fullPrice();
  if (selectedTowUser && this.user.userLocationLat && this.user.userLocationLng && fullPrice && this.vehicles.selectedId) {
    this.towRequest = {
      user: this.auth.currentUser().!id,
      vehicle: this.vehicles.selectedId,
      tow_user: selectedTowUser,
      pickup_lat: this.user.userLocationLat,
      pickup_long: this.user.userLocationLng,
      pickup_note: this.pickupAddress,
      dropoff_lat: this.dropoffLat,
      dropoff_long: this.dropoffLng,
      status: "awaiting response",
      price: fullPrice
    }
  }
}

this.towRequests.addTowRequest(this.towRequest).subscribe({
  next: () => {
    this.auth.isRequesting.set(true);

    const selectedId = this.towUser.selectedTowUser();
    if (selectedId != null) {
      this.towUser.selectedTowUser.set(selectedId);
    }

    this.closeDialog()
  }
})
}

```

34. Kérés elküldésére szolgáló metódus

Sikeres kérés esetén a komponens bezáródik, és a rendszer frissíti az aktuális állapotot, jelezve, hogy a felhasználónak aktív vontatási kérelme van.

A TowRequestEndWindow komponens is egy felugró ablakban (dialog) megjelenő komponens, amely a vontatási folyamat lezárásáért felelős. A komponens megjeleníti a vontatásban részt vevő felhasználó és autómentő adatait, valamint a végleges árat és a megtett távolságot. A folyamat lezárása mindkét fél megerősítéséhez kötött, így a felhasználónak és az autómentőnek is külön meg kell erősítenie a vontatás befejezését és a fizetést. A megerősítés állapotát 5 másodpercenként lekéri a pollRequest metódussal, és ez alapján frissíti az oldalt.

```
pollRequest() {
  if (this.request.id) {
    this.towRequestService.getTowRequestById(this.request.id).subscribe({
      next: (request) => {
        if (request.user_confirmed === 1) {
          this.userConfirmed.set(true)
          console.log('interval set userConfirmed to true')
        }
        if (request.tow_user_confirmed === 1) {
          this.towUserConfirmed.set(true);
          console.log('interval set towUserConfirmed to true')
        }
        if (this.towUserConfirmed() && this.userConfirmed()) {
          const updatedRequest = {
            status: 'completed'
          };
          if (!this.request?.id) return;
          this.towRequestService.updateTowRequest(this.request.id, updatedRequest).subscribe()
        }
        this.request = request
      }
    })
  }
}
```

35. A pollRequest függvény

A megerősítés gombra kattintva frissíti az adatbázist a komponens a confirmEndAndPayment metódussal, és ha a másik fél már megerősítette, átváltja a vontatási kérés állapotát befejezettre. Frissítésnél és lekérésnél a szükséges adatok signalokban vannak tárolva.

```

this.towRequestService.updateTowRequest(this.request.id, updatedRequest).subscribe({
  next: () => {
    this.loading.set(false);

    if (this.towUserConfirmed() && this.userConfirmed()) {
      updatedRequest = {
        status: 'completed',
        tow_user_confirmed: 1,
        user_confirmed: 1
      };

      if (!this.request?.id) return;

      this.towRequestService.updateTowRequest(this.request.id, updatedRequest).subscribe({
        next: () => {
          console.log("status set to complete")
        }
      })
    }
  }
});

```

36. Megerősítés és állapot frissítése a *confirmEndAndPayment* függvényben

A SubmitRatingWindow komponens is egy felugró ablakban (dialog) megjelenő komponens, amely az autómentő értékelésére szolgál a vontatás befejezése után. A komponens lehetőséget biztosít a felhasználó számára, hogy csillagok segítségével értékelje az autómentőt, valamint opcionálisan szöveges visszajelzést is megadhat.

A kiválasztott értékelést a komponens egy RatingPost objektumba rendezi, majd a RatingService segítségével elküldi a backendnek. Sikeres beküldés után a komponens bezáródik, és visszatér az előző nézethez.

```

submitRating() {
  if (this.selectedStars() === 0) return;

  const user = this.auth.currentUser()?.id
  if(!user) return;

  this.loading.set(true);

  const ratingData : RatingPost = {
    user: user,
    tow_user: this.towUserId,
    tow_request: this.towRequestId,
    rating: this.selectedStars(),
    text: this.reviewText.trim() || undefined
  };

  this.ratingService.addRating(ratingData).subscribe({
    next: () =>{
      console.log("rating submitted");
    }
  })

  this.loading.set(false);
  this.dialogRef.close(ratingData);
}

```

37. Az értékelés elküldése a submitRating függvény segítségével

ProfilPanelComp

A webalkalmazás az Angular Router segítségével kezeli az oldalak közötti navigációt. A routing meghatározza, hogy az adott URL milyen komponenst jelenítsen meg. A '/' főoldal, a térképes felületet jeleníti meg (TowMapPage komponens). A '/profile' a felhasználó fiók és beállítások oldalt jeleníti meg (ProfilPanelComp komponens).

```

export const routes: Routes = [
  {
    path: '',
    component: TowMapPage
  },
  {
    path: 'profile',
    component: ProfilePanelComp
  }
];

```

38. Route-ok az app.route.ts-ben

A weboldal reszponzív, telefon és tablet képernyőméreteken is megfelelően jelenik meg és használható marad.

 Helyadat lekérése

Cím beírása

 Cím keresése

km 

Km-en belül

 Keresés



 Leaflet | © OpenStreetMap contributors (ODbL) - Routing by OSRM

39. Főoldal mobil nézetben

Helyadat lekérése

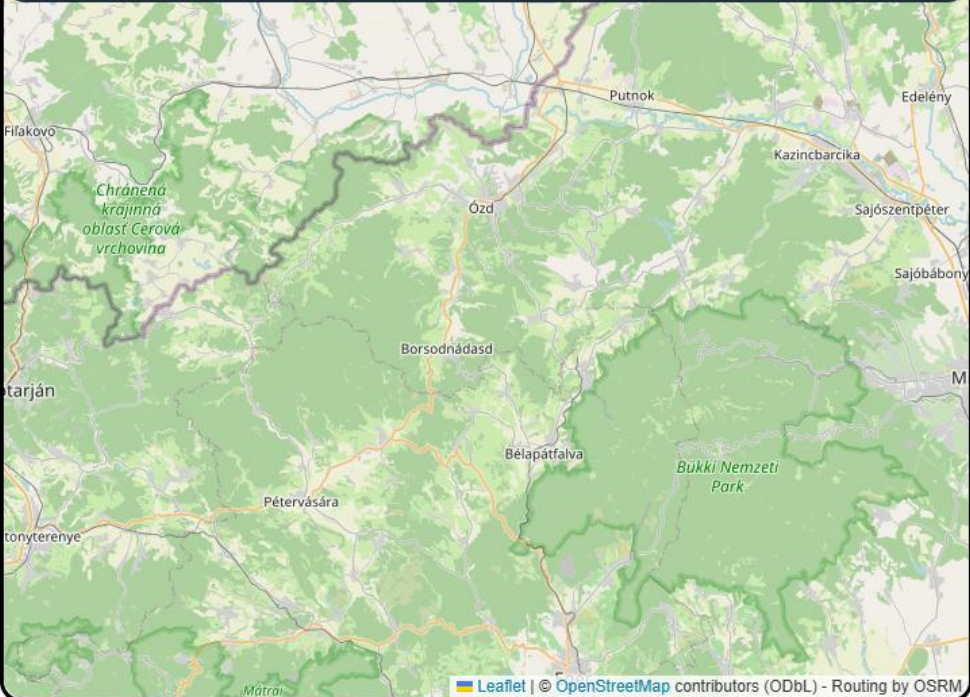
Cím beírása

Cím keresése

km

Km-en belül

Keresés



Map showing the area around Ózd, Putnok, and Edeleny, including the Bükk National Park (Bukki Nemzeti Park) and the M3 highway. The map is displayed on a tablet screen.

40. Főoldal tablet nézetben



Vontatási Kérés

Autó kiválasztása:

**Suzuki Swift**
Kiválasztva
ASD-124

Autó
törlése

+

Új autó hozzáadása

Tartózkodási hely pontosítása (az autómentő számára):

pl. Ózd, Kossuth Lajos u. 83

Lerakási cím (utca) megadása:

pl. Ózd, Március 15 utca / Ózd,

Cím keresése

Végleges Ár: - Ft

- Km
1200 Ft / Km

Mégsem

Kérés küldése

Leaflet | © OpenStreetMap contributors (ODbL) - Routing by OSRM

41. Vontatás kérése mobil nézetben

 QUICKTOW

testUser

Vontatási Kérés

Autó kiválasztása:

 **Suzuki Swift** Kiválasztva
ASD-124

Autó törlése

 Új autó hozzáadása

Tartózkodási hely pontosítása (az autómentő számára):

pl. Ózd, Kossuth Lajos u. 83

Lerakási cím (utca) megadása:

pl. Ózd, Március 15 utca / Ózd, Göngyösi Gumi Kft

Cím keresése

Végleges Ár: - Ft

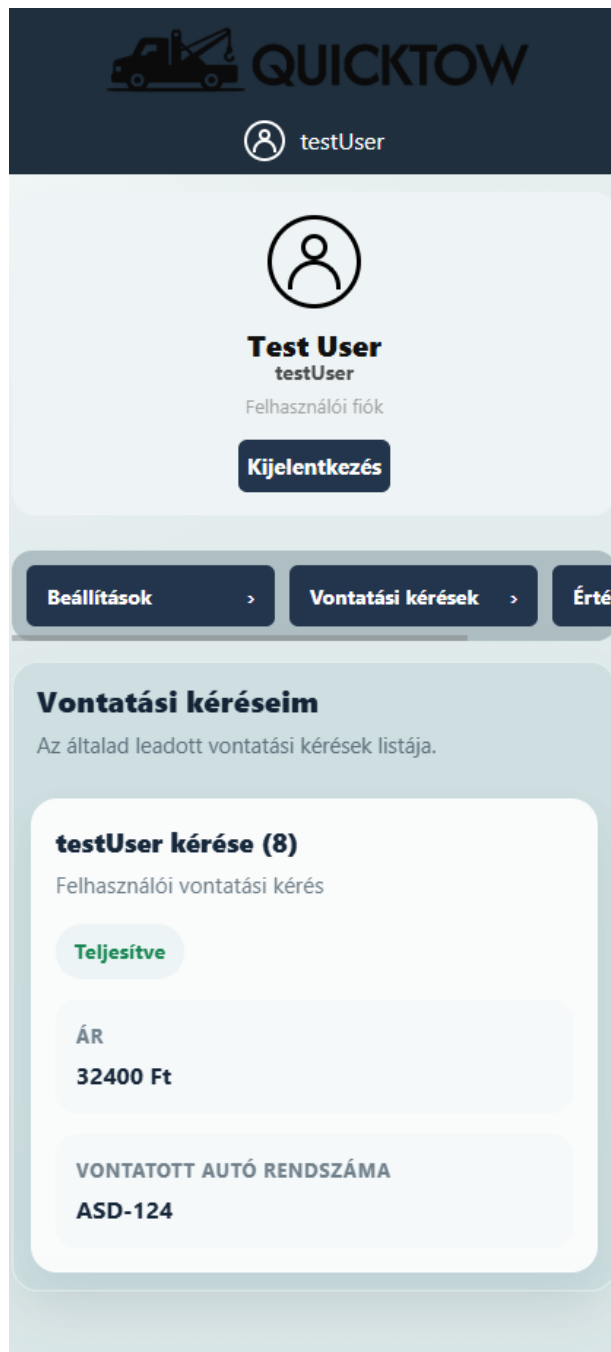
- Km
1200 Ft / Km

Mégsem

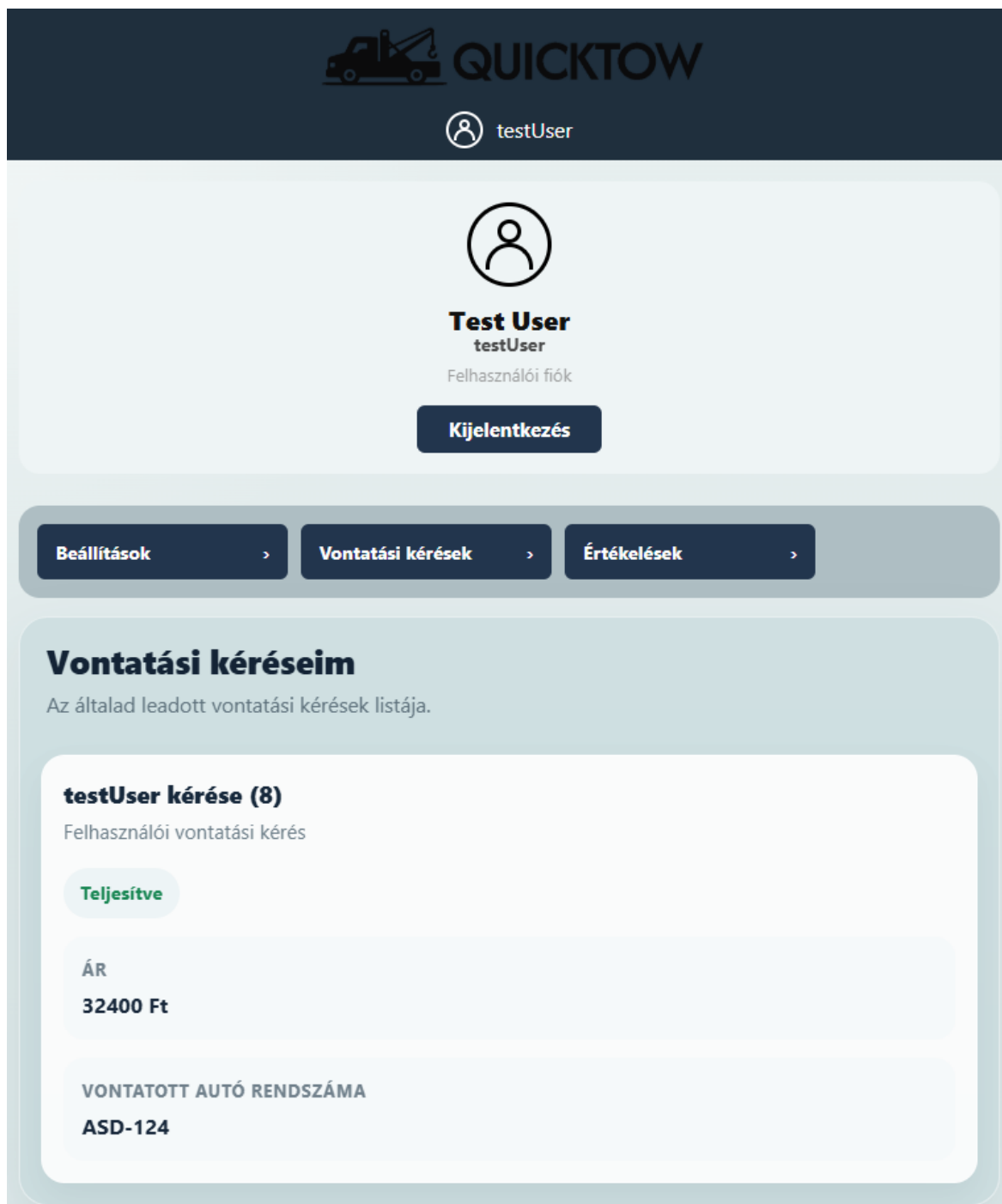
Kérés küldése

Leaflet | © OpenStreetMap contributors (ODbL) - Routing by OSRM

42. Vontatás kérése tablet nézetben



43. Fiók beállítások oldal mobil nézetben



44. Fiók beállítások oldal tablet nézetben

6. Backend

A backend a Laravel keretrendszerben lett megvalósítva. A migrations mappában mind a 6 adattábla létre van hozva az adatbázis struktúrájának megfelelően.

A vehicles táblában az idegen kulcs (user) nullable, ugyanis ha a felhasználó törli a profilját, akkor az a mező automatikusan null értéket kap. Csak ebben a táblában van beállítva soft delete, hogy lehessen törölni járművet felhasználói profilról, de ne vesszen el végleg az adat. A többi olyan táblánál amiből törölhető adat ez nincs beállítva adatbiztonsági okokból. Azaz például ha egy felhasználó törölni szeretné a profilját akkor az adatok véglegesen törlődnek az adatbázisból.

A tow_users adattáblában a price_per_km kap egy 1000-es alapértéket, ez változtatható. A latitude és longitude nullable, nem kell mindig értéket tartalmazniuk, csak ha az autómentő felhasználó aktív / munkában van. A rating mező decimal típusú, mivel az értékelés automatikus van kiszámítva így az osztásból adódóan esetenként sok tizedesjegy lenne látható. A `$table->decimal('rating',3, 2)` használatával csak 2 tizedesjegy lesz bármilyen osztásnál.

```
public function up(): void
{
    Schema::create('tow_users', function (Blueprint $table) {
        $table->id();
        $table->string('last_name', 50);
        $table->string('first_name', 50);
        $table->string('username', 50)->unique();
        $table->string('password');
        $table->string('email', 50)->unique();
        $table->string('phone_number', 50);
        $table->integer('price_per_km')->default(1000);
        $table->double('latitude')->nullable();
        $table->double('longitude')->nullable();
        $table->decimal('rating',3, 2)->default(0);
        $table->integer('rating_count')->default(0);
        $table->string('status', 20)->default('unavailable');
    });
}
```

45. A tow_users tábla struktúrája

A tow_requests, payments és ratings táblában a user és tow_user idegen kulcsai szintűgy nullable-k, hogy törlésük esetén is megmaradjon a rekord többi adata.

A seeder minden táblába feltölt néhány példa adatot, tesztelés szempontjából. A végén meghívja a `refreshRating()` metódust, ami frissíti a `tow_users` adattáblában az értékelések átlagát.

A User modelben a `password` mező a `hidden` kategóriában van, hogy API kéréseknél ne adja vissza a jelszavakat. Összehasonlítani, ellenőrizni ettől ugyan úgy lehet. A `TowUser` modelben a jelszó szintűgy rejtett. A `ratings()` metódus a az egy-sok kötést valósítja meg, azaz egy autómentő felhasználónak több értékelése lehet. A `refreshRating()` az adott autómentőhöz tartozó értékeléseknek számolja ki az átlagát, illetve frissíti le az értékeléseknek a számát. Ha a `ratings()` által visszaadott érték null, azaz nincs értékelés, 0-ás értéket kap az autómentő értékelés mezője. A `scopeWithinRadius` metódus arra szolgál, hogy egy megadott földrajzi koordinátától egy megadott távolságon belül visszaadja az autómentő felhasználókat. A koordinátákat és a sugarat paraméterben kapja meg a controllerből, ezekkel csinál egy SQL kérést. Ez a kérés a Haversine formula alapján kiszámolja az autómentők távolságát a megadott koordinátától. A függvény csak azokat adja vissza, amelyek alacsonyabb távolságra vannak a paraméterben megkapott sugártól.

```

class TowUser extends Model
{
    use HasFactory;
    protected $table = "tow_users";
    public $timestamps = false;
    public $guarded = [];

    protected $hidden = [
        'password'
    ];

    public function ratings()
    {
        return $this->hasMany(Rating::class, 'tow_user');
    }

    public function refreshRating()
    {
        $this->rating = $this->ratings()->avg('rating') ?? 0;
        $this->rating_count = $this->ratings()->count();
        $this->save();
    }

    public function scopeWithinRadius($query, float $lat, float $lng, float $radius)
    {
        return $query->selectRaw("
            tow_users.*,
            (
                6371 * acos(
                    cos(radians(?)) * cos(radians(latitude))
                    * cos(radians(longitude) - radians(?))
                    + sin(radians(?)) * sin(radians(latitude))
                )
            ) AS distance
        ", [$lat, $lng, $lat])
        ->having("distance", "<=", $radius);
    }
}

```

46. A TowUser model és metódusai

A Vehicle, TowRequest, Rating és Payment modellekben a belongsTo() függvény minden külső meg van csinálva, így API kéréskor ki lehet listázni a külső kulcshoz tartozó adatokat is.

```

public function user()
{
    return $this->belongsTo(User::class, 'user');
}

```

47. Példa a belongsTo függvényre a Payment modelben

Mind a 6 modelhez tartozik controller is. A UserControllerben az index metódus visszaadja az összes táblában szereplő rekordot, azaz az összes felhasználó adatát. Mivel ezek között nincs idegen kulcs ezért elég az all() függvényt használni.

```
public function index()
{
    return User::all();
}
```

48. Az index metódus a UserControllerben

A store metódus egy teljesen új rekord tárolására szolgál. Egy Request paramétere van, ebben kapja meg az eltárolni kívánt adatokat. Egy Validatoron belül ellenőrzi, hogy minden kötelező adatot megkapott-e a requestben, illetve hogy milyen típusú (string, numeric, ...) az adat, milyen hosszú és ahol kötelező, hogy egyedi-e az adott mezőben szereplő adat. Ha a validator valamelyik feltételnek nem felel meg, akkor hibát ad vissza JSON-ben, 400-as hibakóddal. A visszadobott hibaüzenet tartalmazza hogy melyik mezők nem feleltek meg a követelményeknek. Ha a request minden adata helyes, akkor egy data változóba kerülnek ezek. Ebből a password (jelszó) mező hash-elve, azaz titkosítva lesz. Ezek után hozza létre az új rekordot és visszaad egy sikeres 201-es kódot az új adatokkal együtt.

```
public function store(Request $request)
{
    $validator = Validator::make(
        $request->all(),
        [
            'last_name' => 'required|string|max:50',
            'first_name' => 'required|string|max:50',
            'username' => 'required|string|max:50|unique:tow_users,username',
            'password' => 'required|string',
            'email' => 'required|email|unique:tow_users,email',
            'phone_number' => 'required|string|max:50',
            'price_per_km' => 'sometimes|required|integer|min:0'
        ]
    );

    if ($validator->fails())
    {
        return response()->json(['Validation failed:' => $validator->errors()], 400);
    }

    $data = $request->all();
    $data['password'] = Hash::make($data['password']);
    $user = TowUser::create($data);
    return response()->json(["User created successfully" => $user],201);
}
```

49. Store metódus a UserControllerben

Az update metódus egy meglévő rekord valamennyi mezőjének változtatására szolgál. Két paramétert kap, egy Requestet és egy id-t. Először find() függvénnyel megkeresi hogy létezik-e rekord a paraméterben megkapott id-vel, és ezt eltárolja a user változóban. Ha nem találta meg a keresett id-t és üres marad a változó, akkor hibát dob, 404-es kóddal.

```
public function update(Request $request, $id)
{
    $user = User::find($id);
    if(is_null($user))
    {
        return response("User not found",404);
    }
}
```

50. Id alapján keresés

Ha nem üres, akkor a validator itt is lefut. Ebben nem kötelező minden adatot újra megadni, mivel már alaphoz van adat a mezőkben így a validator arra szolgál, hogy a megadott adat helyes legyen, szóval ne legyen túl hosszú vagy üres, megfelelő típusú legyen, stb. (ezért csak sometimes|required). A user változóban már benne vannak a módosítani kívánt felhasználó adatai, ezért a request validálása után átírja a módosított mezőket, majd lementi (a jelszó titkosítása itt is megtörténik). A végén egy sikeres üzenetet ad vissza a módosított adatokkal és egy 200-as kóddal.

```

$validator = Validator::make(
    $request->all(),
    [
        'last_name' => 'sometimes|required|string|max:50',
        'first_name' => 'sometimes|required|string|max:50',
        'username' => 'sometimes|required|string|max:50|unique:users,username,' . $user->id,
        'password' => 'sometimes|required|string',
        'email' => 'sometimes|required|email|unique:users,email,' . $user->id,
        'phone_number' => 'sometimes|required|string|max:50'
    ]
);
if ($validator->fails())
{
    return response()->json(['Validation failed:' => $validator->errors()], 400);
}

$user->fill($request->only([
    'last_name',
    'first_name',
    'username',
    'email',
    'phone_number'
]));

$user->password = Hash::make($request->password);

$user->save();

return response()->json(['User updated successfully'=>$user],200);
}

```

51. Az update metódus a UserControllerben

A destroy metódus kitörli a paraméterben megkapott id-vel rendelkező felhasználót. Először megkeresi az adott rekordot, amit eltárol egy változóban. Hibát dob ha üres, azaz ugyan úgy működik mint az update metódusnál, ugyan azt a hibát dobja. Ha nem üres a változó akkor törli a rekordot, és sikeres üzenetet küld vissza 200-as kóddal.

```

public function destroy($id)
{
    $user = User::find($id);
    if(is_null($user))
    {
        return response("User not found",404);
    }
    $user->delete();
    return response("User deleted successfully",200);
}

```

52. A destroy metódus a UserControllerben

A getByld metódus csak a paraméterben megkapott id-vel rendelkező rekordot adja vissza. A rekord keresése a find() függvénnyel ugyan úgy működik mint az előző metódusokban. Ha megtalálta az adott id-t akkor visszaadja azt a rekordot JSON-ben, 200-as kóddal.

A TowUserControllerben az index, store, update, destroy és getByld metódusok megegyeznek az UserControllerben lévőkkel, csak a vizsgált mezők nevei különböznek.

A getStatus az paraméterben megkapott állapot (offline, busy, available) alapján szűri az autómentőket. Az allowed változóban vannak eltárolva egy tömbben a megfelelő szűrési típusok, azaz ha az API kérésben nem helyes van megadva az állapot neve hibát dob JSON-ben, 400-as HTTP kóddal.

```
$allowed = ['offline','busy','available'];  
  
if(!in_array($status,$allowed))  
{  
    return response('Invalid status',400);  
}
```

53. Engedélyezett állapotok és annak vizsgálata

A towUser változóba lekéri azokat az autómentőket amelyeknek az állapota megegyezik a kért állapottal. Ha nincs ilyen, akkor külön üzenetet ad vissza, 200-as hibakóddal, mivel a kérés helyes volt, csak nincs éppen annak megfelelő rekord. Ha talált a kérésnek megfelelő rekordot, akkor azokat JSON-ben visszaadja, 200-as kóddal.

```
$towUser = TowUser::where('status','=', $status)->get();  
if($towUser->isEmpty())  
{  
    return response('No tow users found',200);  
}  
return response()->json($towUser,200);
```

54. Kért állapot alapján szűrés és visszaadás

A `getMinRating` a paraméterben megadott értékeléstől magasabb értékelésű autómentőket adja vissza. Ugyan olyan módon működik mint a `getStatus`, csak mást vizsgál meg. Ha talál a kérésnek megfelelő rekordokat, akkor azokat adja vissza JSON-ben 200-as kóddal.

```
$towUser = TowUser::where('rating','>=',$rating)->get();
```

55. Értékelés alapján szűrés

A `getWithinRadius` az URL paraméterekben megadott földrajzi koordinátától a szintúgy URL-ben megadott sugáron belüli autómentőket adja vissza. Itt is van `Validator`, ami a kapott paramétereket vizsgálja, hogy ne legyen üresek, illetve számok legyenek. Ha ez nem így van, akkor hibaüzenetet dob a validator hibáival és 400-as kóddal.

A `lat`, `long` és `radius` változóiban számokká alakítja és eltárolja az URL-ből kapott paramétereket. Egy új `towUser` változóban meghívjuk a `TowUser` modelben lévő `scopeWithingRadius` metódust, aminek paraméternek átadjuk a 3 előbb eltárolt változót. Ez visszaadja az adott távolságon belül lévő autómentőket. Ha nem ad vissza semmit, azaz nincs a kérésnek megfelelő rekord, akkor 204-es (No content) üzenetet küld vissza. Ha a `towUser` változó nem üres, akkor visszaadja a távolságon belül lévő autómentőket JSON-ben, 200-as kóddal.

```
public function getWithinRadius(Request $request)
{
    $validator = Validator::make($request->all(),[
        'lat' => 'required|numeric',
        'long' => 'required|numeric',
        'radius' => 'required|numeric|min:1|max:100'
    ]);
    if ($validator->fails())
    {
        return response()->json(['Validation failed:' => $validator->errors()], 400);
    }

    $lat = (float) $request->query('lat');
    $long = (float) $request->query('long');
    $radius = (float) $request->query('radius');

    $towUsers = TowUser::withinRadius($lat,$long,$radius)->get();

    if($towUsers->isNotEmpty())
    {
        return response()->json($towUsers,200);
    }
    return response("No tow users found withing the given radius",204);
}
```

56. A `getWithinRadius` metódus a `TowUserController`ben

A TowRequestControllerben az index metódus visszaad minden vontatási kérést az adatbázisból. Az idegen kulcsokhoz (user, vehicle és towUser) tartozó más táblában lévő rekordok adatait is kiírja a with() függvény segítségével.

```
public function index()
{
    return TowRequest::with('user','vehicle','towUser')->get();
}
```

57. Az index metódus a TowRequestControllerben

A store, és getByld metódus hasonló módon működik mint a többi controllerben, csak más mezők ellenőrzésével a validatorban. Az update-ben az allowed változóban a változtatható adatok vannak eltárolva. Ez ebben az esetben csak a status mező, ugyanis csak ezt kell változtatni, a többi nem módosítható. A validator is csak ezt az egy mezőt ellenőrzi. Ha a requestben más adatot is kap, nem dob hibát, csak szimplán azokat az adatokat változtatja meg ami az allowed változóban benne van.

```
$allowed = [
    'status'
];

$validator = Validator::make($request->only($allowed), [
    'status' => 'sometimes|required|string|max:20'
]);

if ($validator->fails())
{
    return response()->json(['Validation failed:' => $validator->errors()], 400);
}

$towRequest->fill($request->only($allowed));
$towRequest->save();
```

58. Az update metódus része a TowRequestControllerben

A TowRequest id alapú lekérés mellett a user illetve a tow_user id-jéhez tartozó vontatási kéréseket is lekérhetjük a getBy metódus segítségével. Ez két paramétert kap: típus és id. A típus a kérés típusa, azaz hogy egy felhasználó vagy autómentő id-jére keresünk-e rá. Itt is van allowed változó, ami az engedélyezett típusokat tartalmazza. Ha az api végpontban hibás típust adunk meg (ami nincs benne a változóban) akkor 400-as kóddal hibát dob. Bevezetünk egy userExists változót, ami majd annak az ellenőrzésére szolgál hogy a keresett id-jű

felhasználó vagy autómentő létezik-e. Ez alapból null értéket kap. Ha a find metódus után is null marad az értéke, akkor a keresett rekord nem létezik, hibát ad vissza 404-es kóddal.

```
$userExists = null;
if($type == 'user')
{
    $userExists = User::find($id);
    if(is_null($userExists))
    {
        return response('User not found',404);
    }
}
else if ($type == "tow_user")
{
    $userExists = TowUser::find($id);
    if(is_null($userExists))
    {
        return response('Tow user not found',404);
    }
}
```

59. Az adott típusú rekord keresése.

Ha megtalálta a keresett rekordot, a where függvény segítségével megkeresi a kért típusú és id-jű rekordokat és eltárolja ezeket egy változóban. Ha nem tartozik a kérésben szereplő id-hez egy rekord sem, akkor egy „No tow requests found” üzenettel és 200-as kóddal, mivel a kérés helyes és helyes a típus és az id is, csak nem tartozik hozzá rekord. Ha nem üres a változó, visszaadja a rekordokat JSON-ben, 200-as kóddal. Röviden így le tudjuk kérni az 'x' id-jű autómentőhöz vagy felhasználóhoz tartozó összes vontatási kérelmet.

A VehicleControllerben is tartalmazza az index, store, destroy és getById metódusokat, ezek ugyan úgy működnek mint a többi modelnél. Ezek mellett van egy getByUser metódus, amivel az adott felhasználóhoz tartozó összes járművet lekérhetjük. Először megvizsgálja, hogy az adott user id-vel létezik-e felhasználó. Ha nem, hibaüzenetet dob 404-es kóddal. Ha létezik a felhasználó, de nem tartozik hozzá jármű, akkor „No vehicles found” üzenetet ad vissza, 200-as kóddal. Ha van felhasználóhoz tartozó jármű / járművek, akkor ezt adja vissza JSON-ben 200-as kóddal

```
$vehicles = Vehicle::where('user','=', $id)->get();
if($vehicles->isEmpty())
{
    return response('No vehicles found',200);
}

return response()->json($vehicles,200);
```

60. A getByUser metódus része a VehicleControllerben

A PaymentControllerben is megtalálhatóak az index, store, getById és getBy metódusok. A getBy-ban itt tow_request alapján is lehet szűrni, szóval az allowed változó ezt is tartalmazza.

```
public function getBy($type, $id)
{
    $allowed = ['user', 'tow_user', 'tow_request'];
```

61. A getBy metódus része a PaymentControllerben

A RatingController szintűgy tartalmazza az index, store, destory, getById és getBy metódusokat. Itt a getBy-ban a szűrés csak user és tow_user alapján lehetséges.

A TowUserControllerben, és a UserControllerben megtalálható login metódus felel mindazért, hogy a felhasználó könnyedén be tudjon jelentkezni a felhasználói fiókjába. A frontendben eldől az, hogy a felhasználó éppen egy vontatósként jelentkezik be, vagy csak egy sima felhasználóként. A kérés végpontját ott szabja meg a rendszer, és elküldi az adott végpontra a kérést. Fontos, hogy a backend vizsgálja meg a jelszót úgy, hogy átalakítja HASH formába, és ha nincs eltérés akkor visszatér a metódus a fiók adataival.

```
public function login(Request $request)
{
    $validator = Validator::make(
        $request->all(),
        [
            'username' => 'required|string',
            'password' => 'required|string'
        ]
    );

    if ($validator->fails()) {
        return response()->json($validator->errors(), 400);
    }

    $user = TowUser::where('username', $request->username)->first();

    if (is_null($user)) {
        return response()->json(["message" => "User not found"], 404);
    }

    if (!Hash::check($request->password, $user->password)) {
        return response()->json(["message" => "Wrong password"], 401);
    }

    return response()->json([
        "message" => "Login successful",
        "user" => $user
    ], 200);
}
```

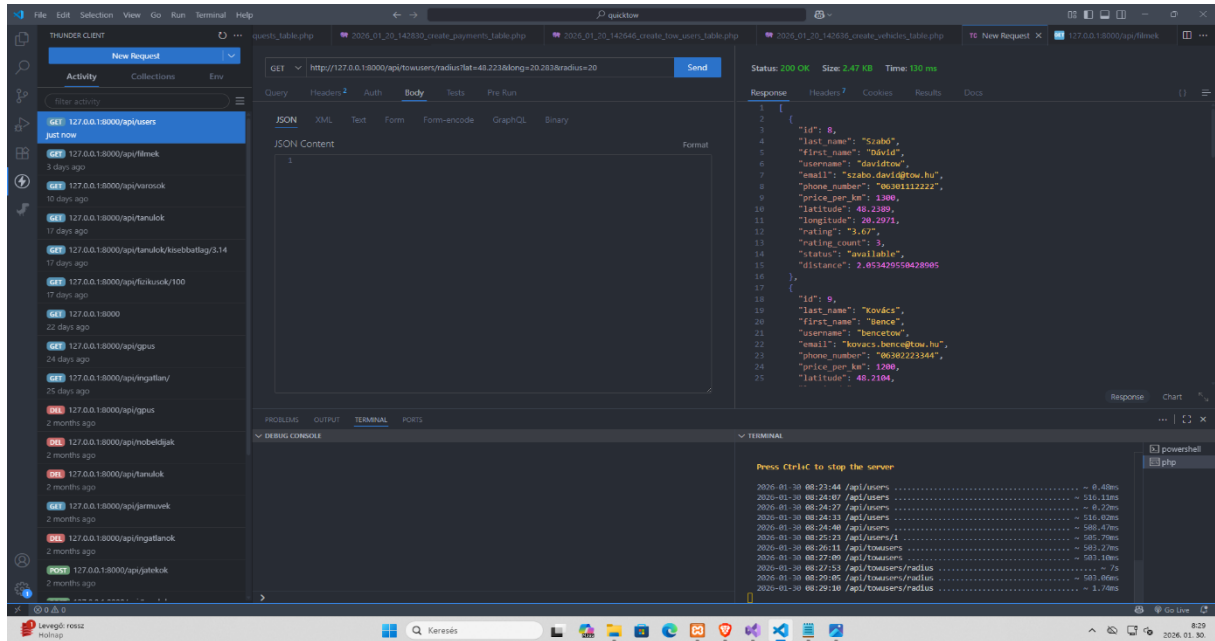
51. A login metódus a UserControllerben

7. Tesztelés

A backend tesztelés szekcióban minden fontosabb metódusról egy sikeres és egy sikertelen lekérés olvasható, a sikertelen kéréseknél magyarázat található.

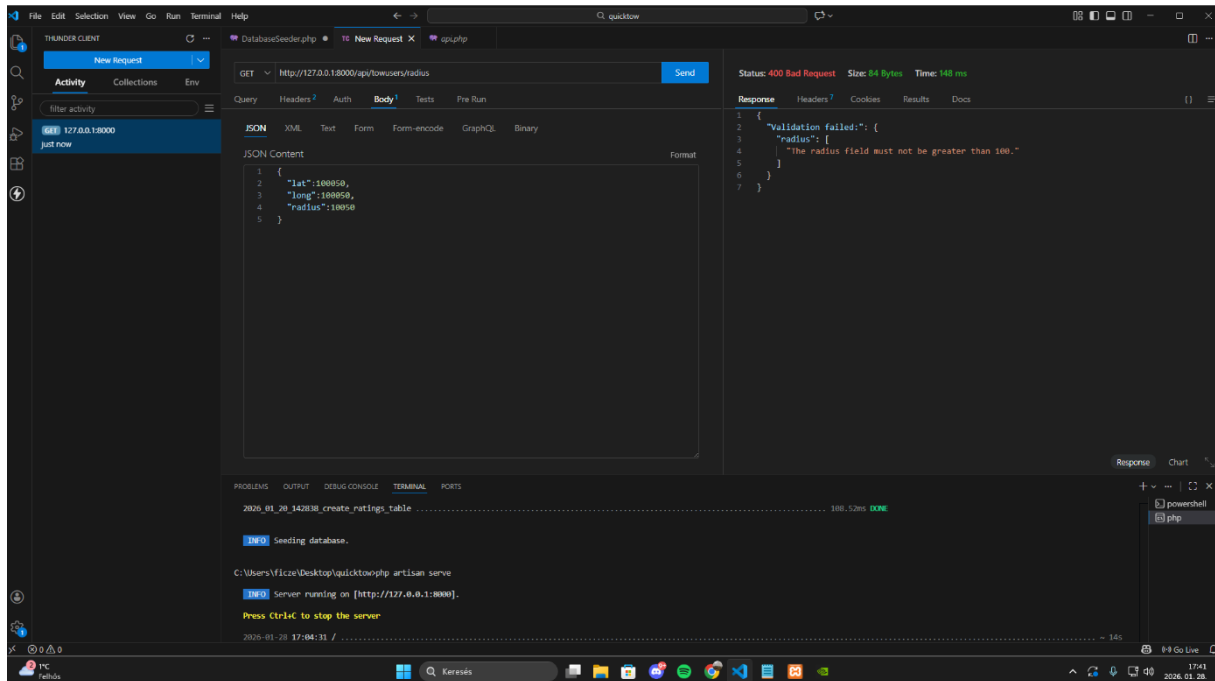
1. towusers

- sikeres Radius() lekérés



62. Sikeres Radius lekérés

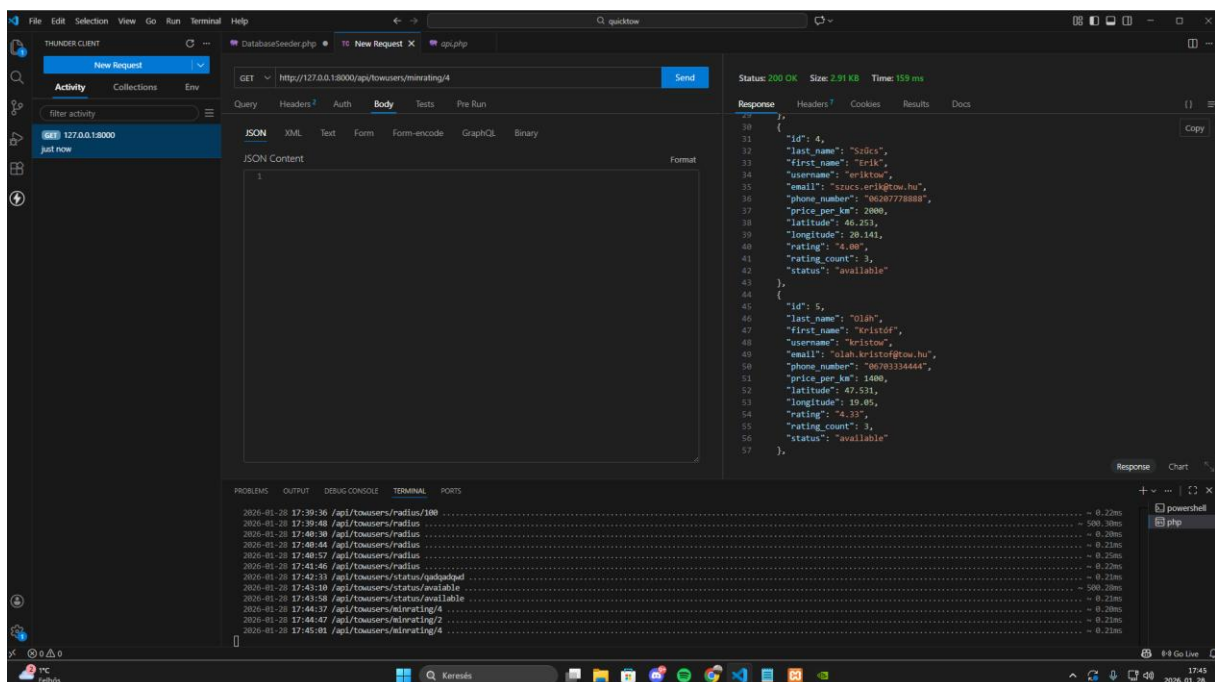
- sikertelen Radius() lekérés



63. Sikertelen Radius lekérés

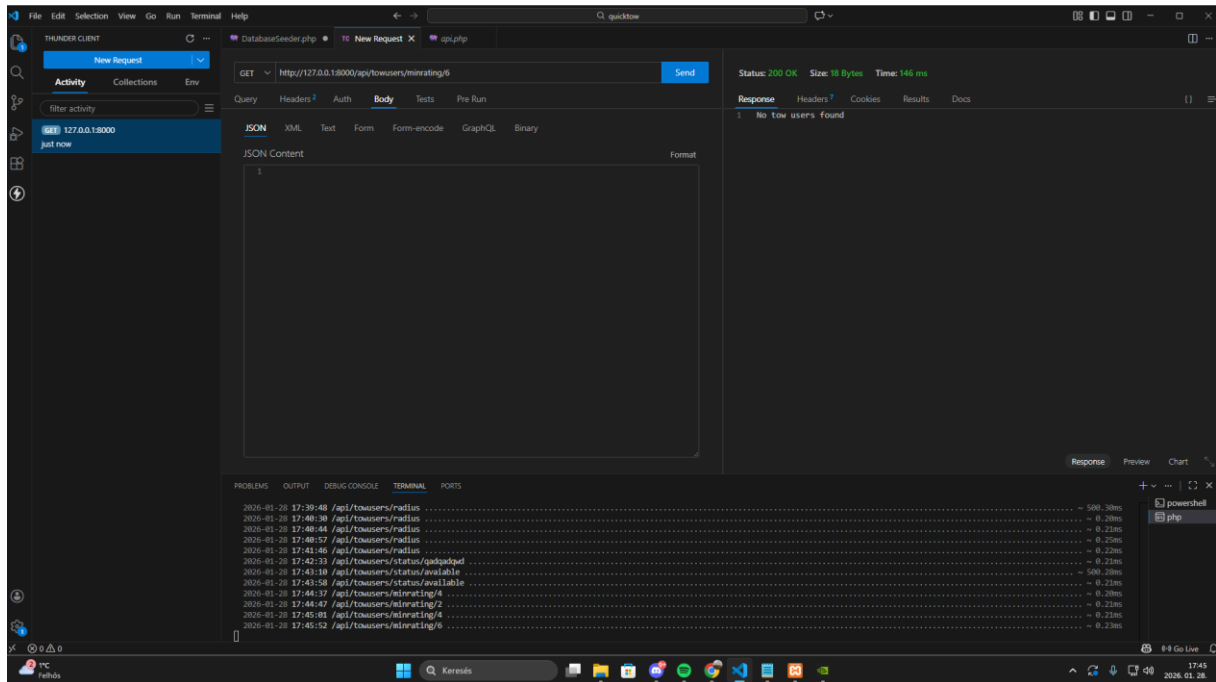
- ha a felhasználó nem ad meg, vagy nagyobb mint 100-as értéket ad meg a lat és a long értékhez, 400-as hibakóddal hibát kap vissza.

- sikeres MinRating() lekérés



64. Sikeres MinRating lekérés

- sikertelen MinRating() lekérés

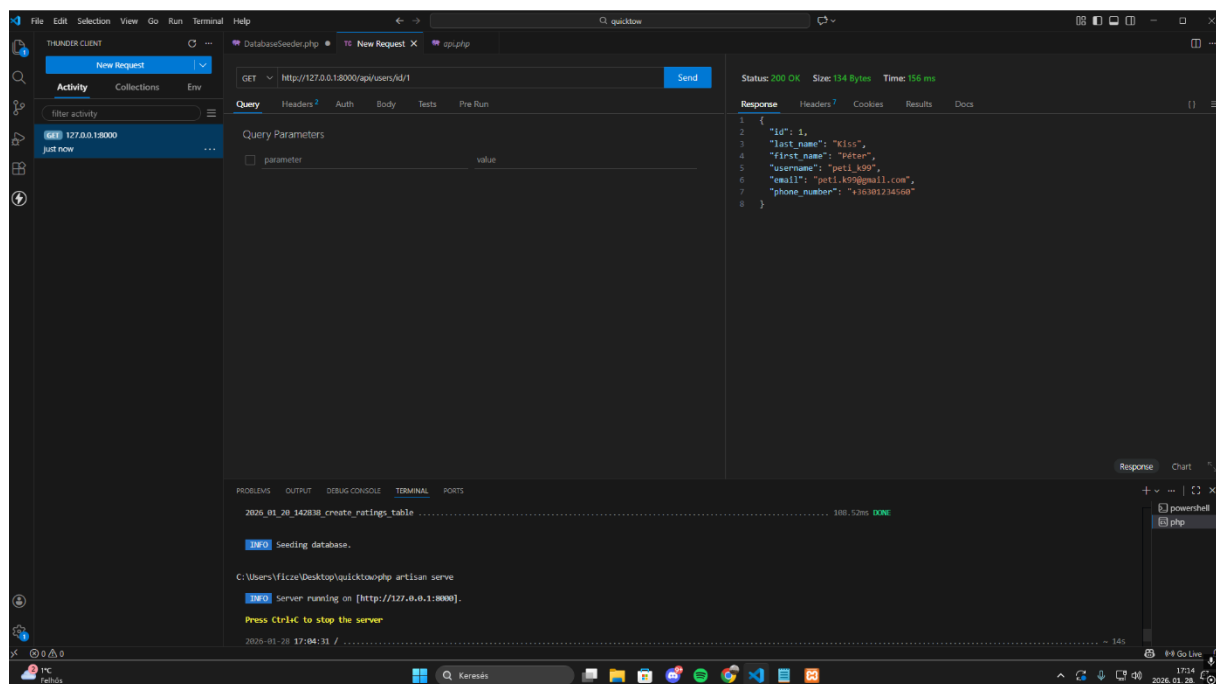


65. Sikertelen MinRating lekérés

- ha a keresésnek egyetlen vontatós sem felel meg, akkor a visszatérő adatok helyett egy hibát kap.

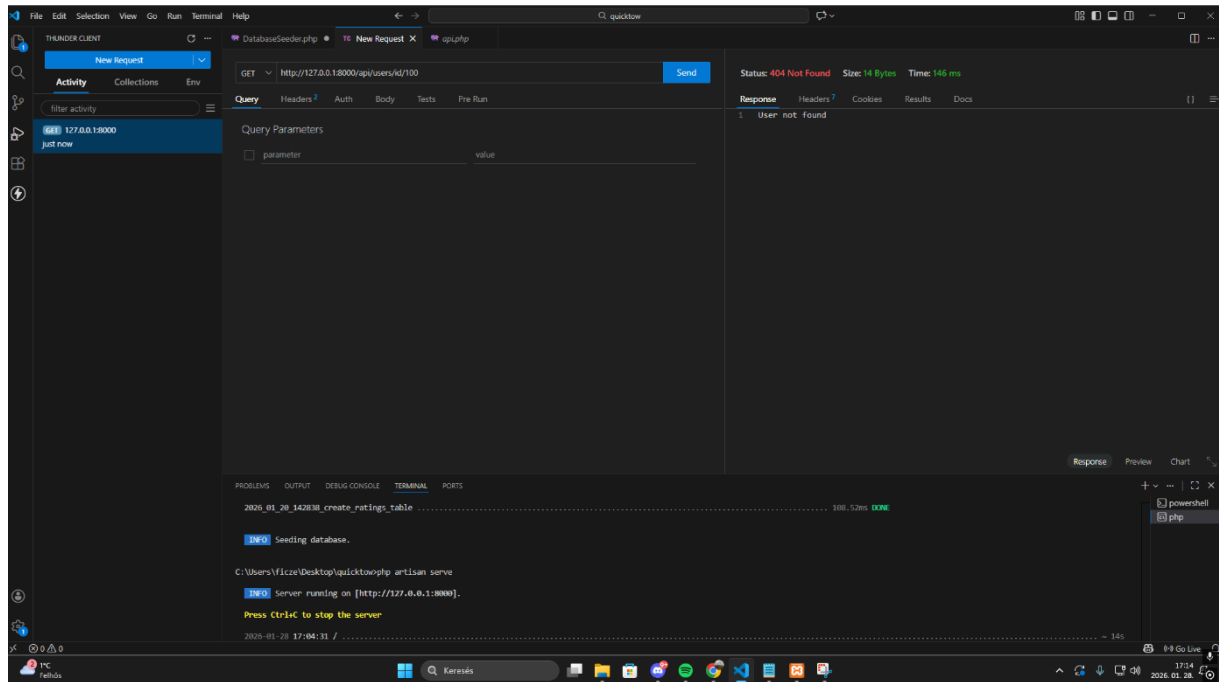
2.users

- sikeres GetById lekérés



66. Sikeres GetById lekérés

- sikertelen GetById lekérés

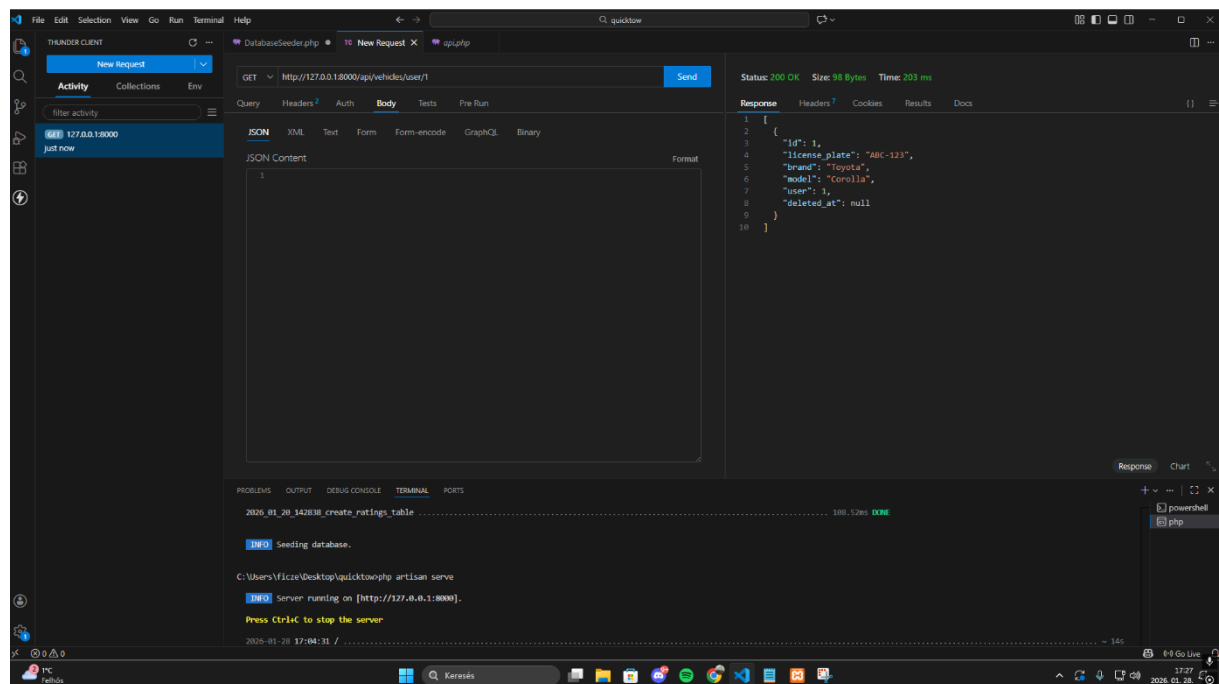


67. Sikertelen GetById lekérés

- ha a felhasználó nem ad meg id-t, vagy olyan id-t ad meg ami nincs az adatbázisban, visszatéréskor „User not found” hibát kap vissza.

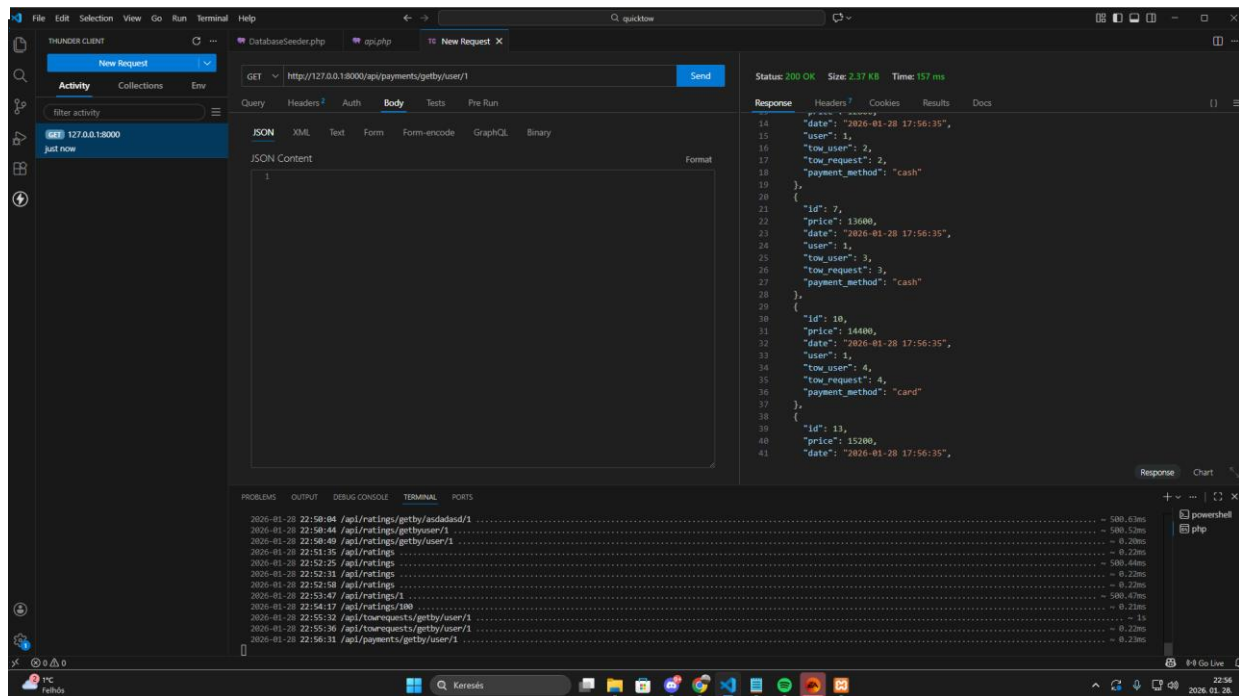
3.vehicles

- sikeres GetByUserId() lekérés



68. Sikeres GetByUserId lekérés

- sikertelen GetById() lekérés

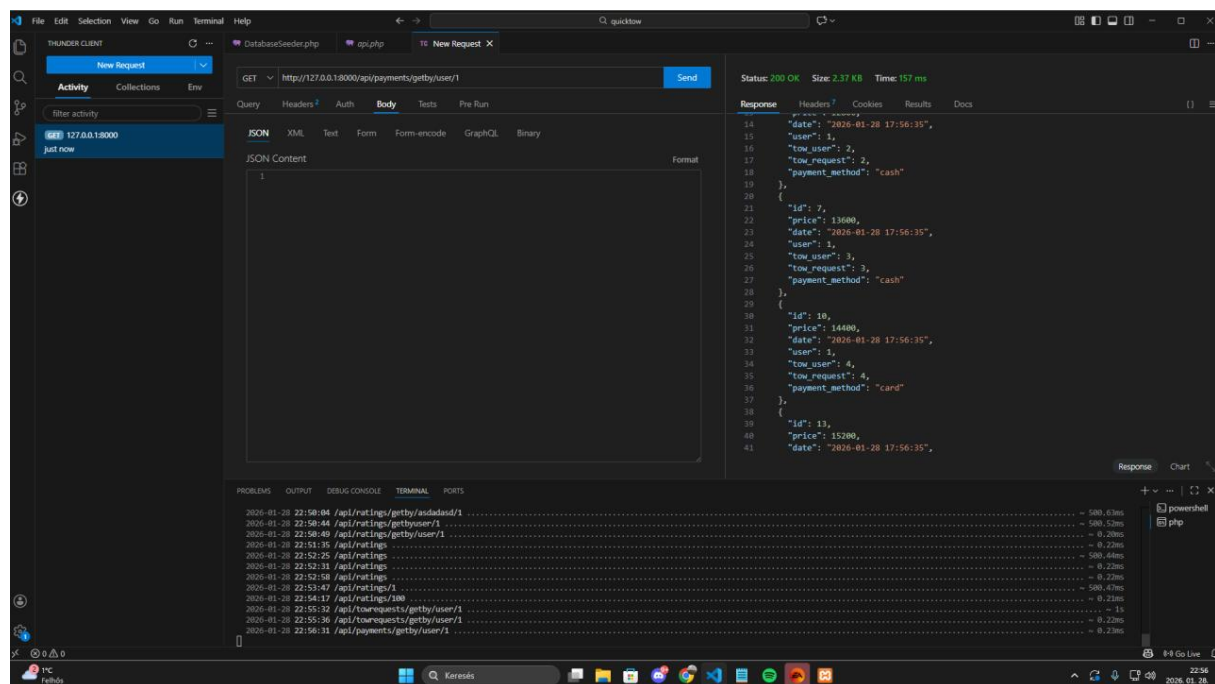


69. Sikertelen GetById lekérés

- ha a felhasználó egy olyan user autót akarja lekérdezni, akinek még a rendszerben nincs autója, hibát kap.

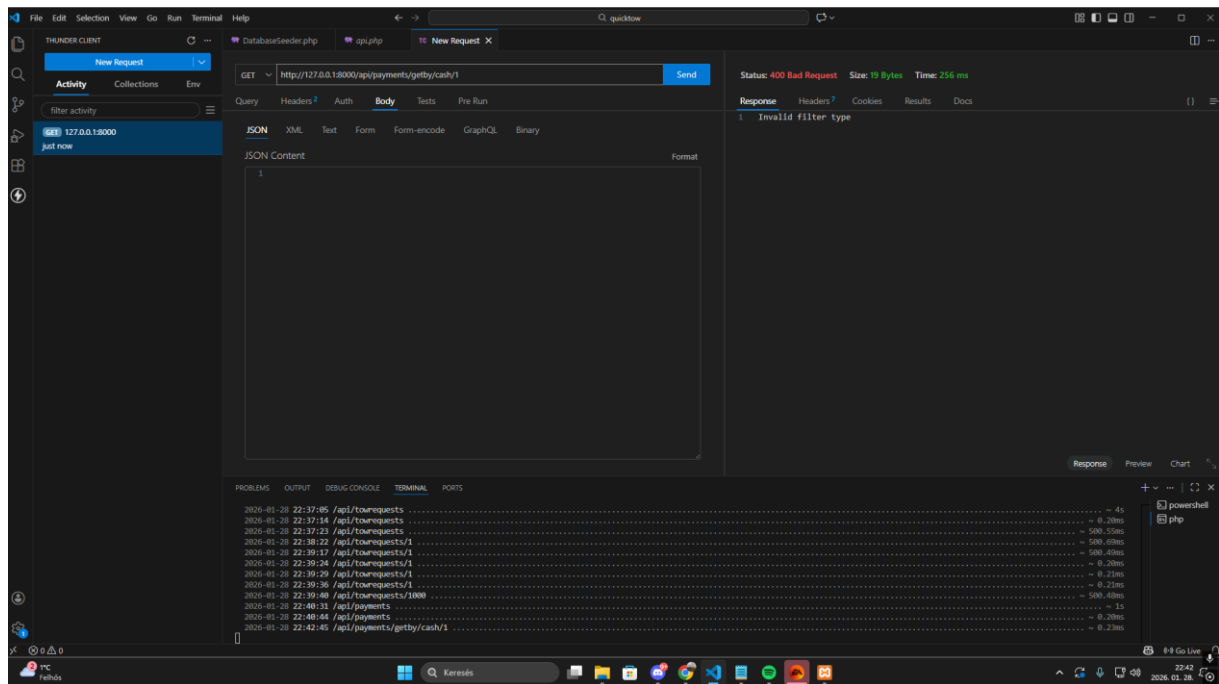
4.payments

- sikeres GetById() lekérés



70. Sikeres GetById lekérés

- sikertelen GetBy() lekérés

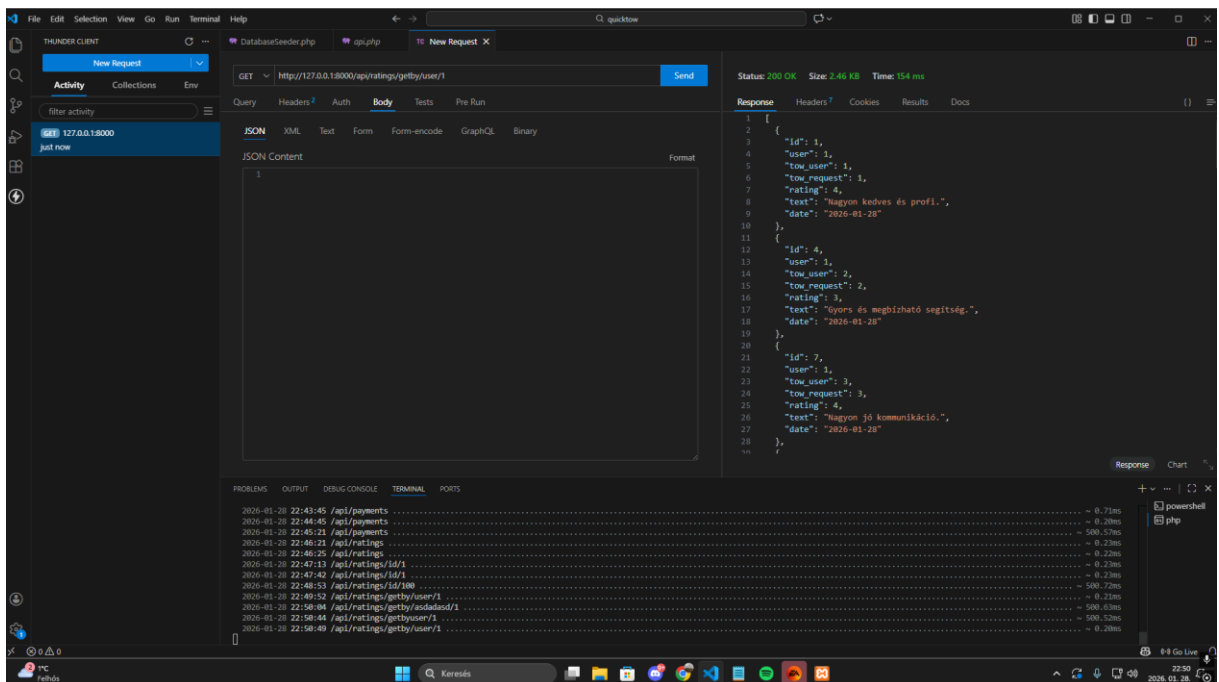


71. Sikertelen GetBy lekérés

- ha a felhasználó az előre megadott állapotok helyett valami mást ad meg, akkor a visszatérésben hibakód jön.

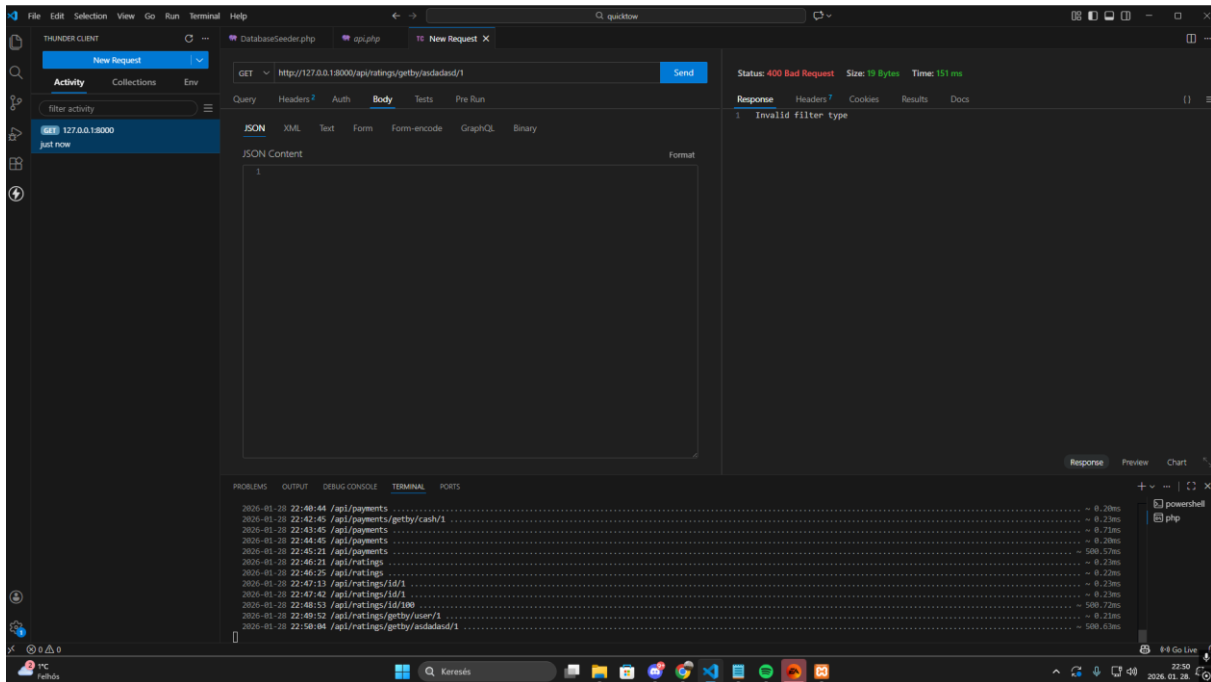
5.ratings

- sikeres GetBy() lekérés



72. Sikeres GetBy lekérés

- sikertelen GetBy() lekérés

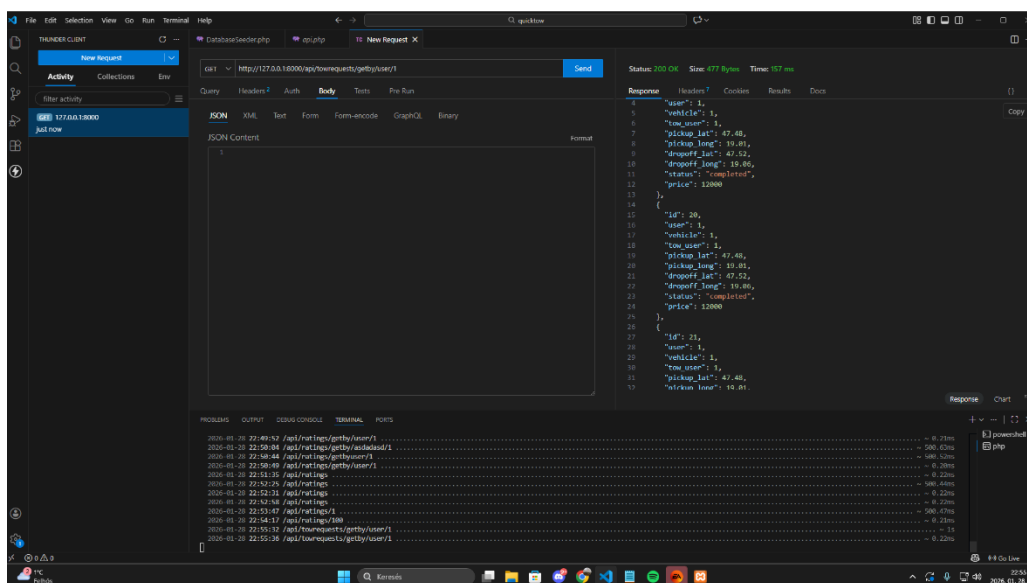


73. Sikertelen GetBy lekérés

- ha a felhasználó hibásan kérdezi le az adatokat, akkor hibakóddal tér vissza a lekérdezés.

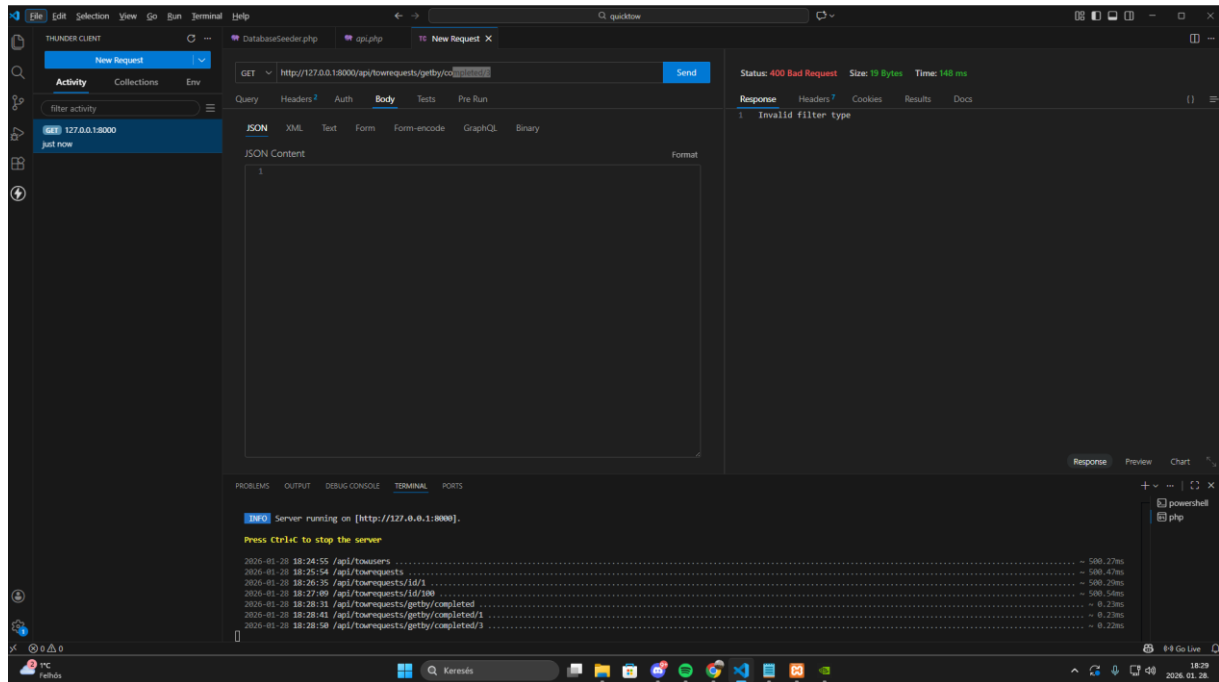
6.towrequests

- sikeres GetBy() lekérés



74. Sikeres GetBy lekérés

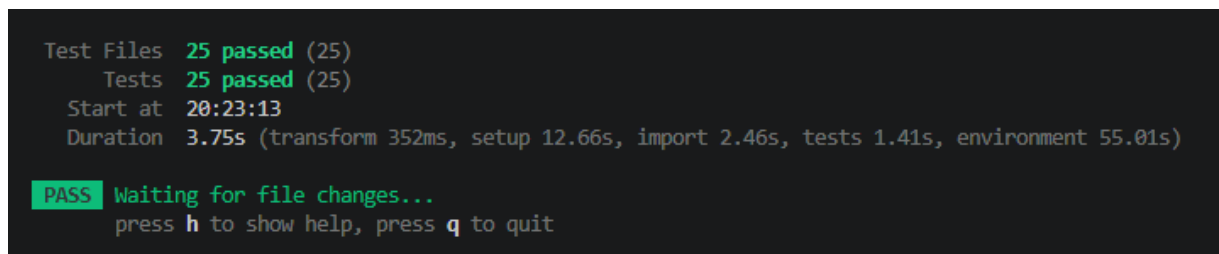
- sikertelen GetBy() lekérés



75. Sikertelen GetBy lekérés

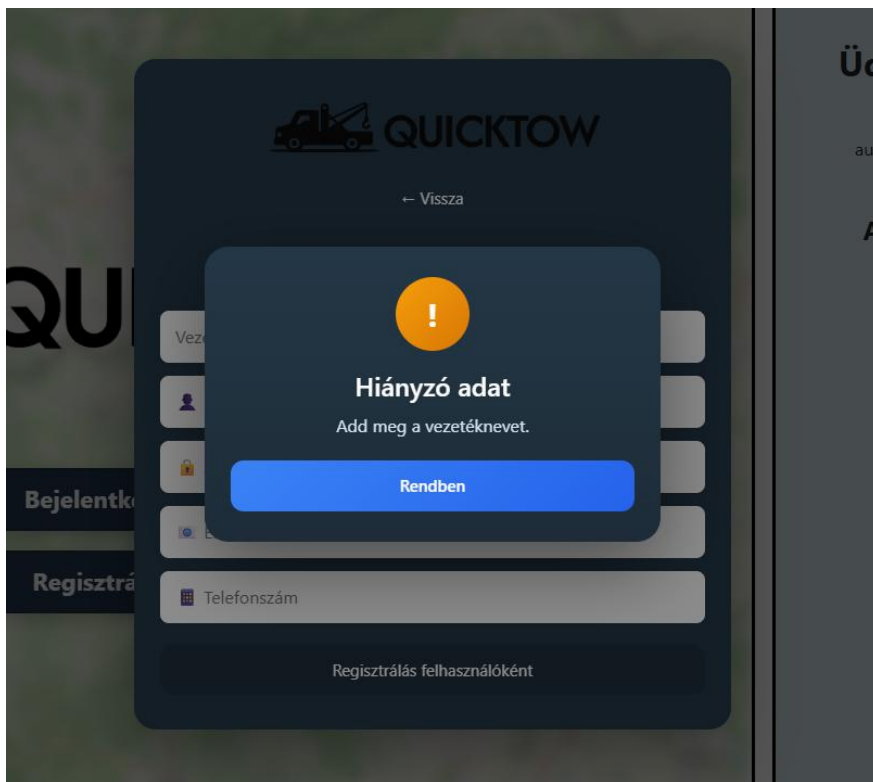
- ha a felhasználó elrontja a lekérdezést, a visszatérésben hibát kap vissza.

Frontend tesztelése ng test használatával:

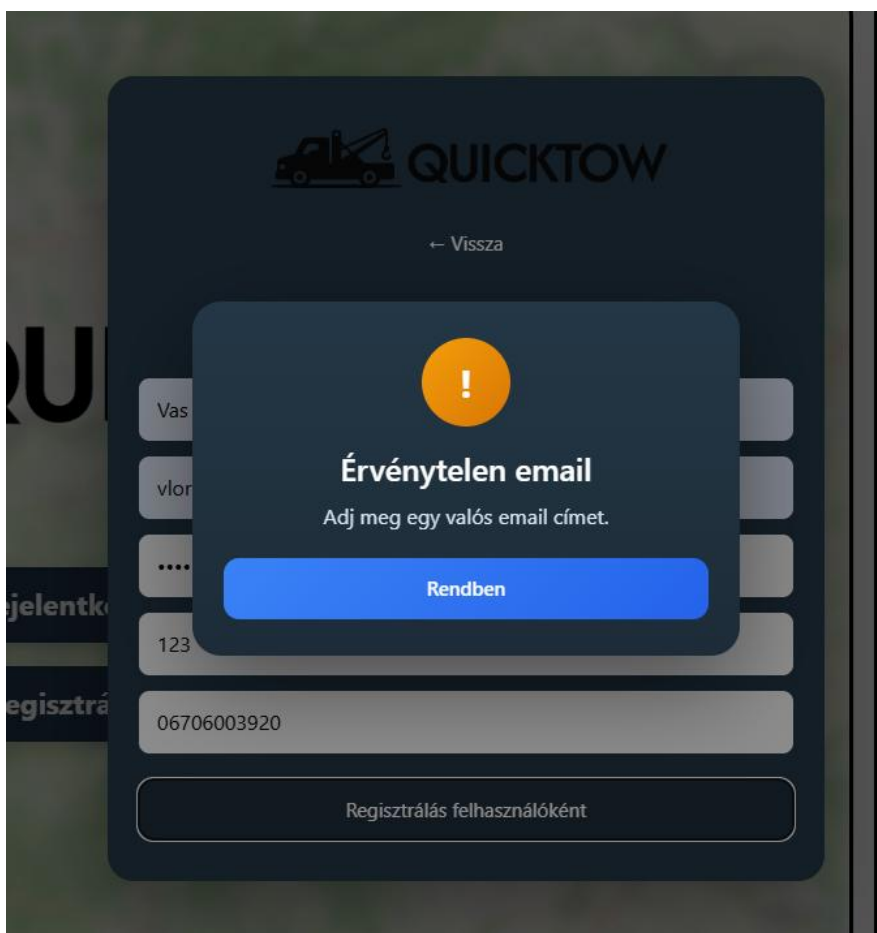


76. Ng test eredménye

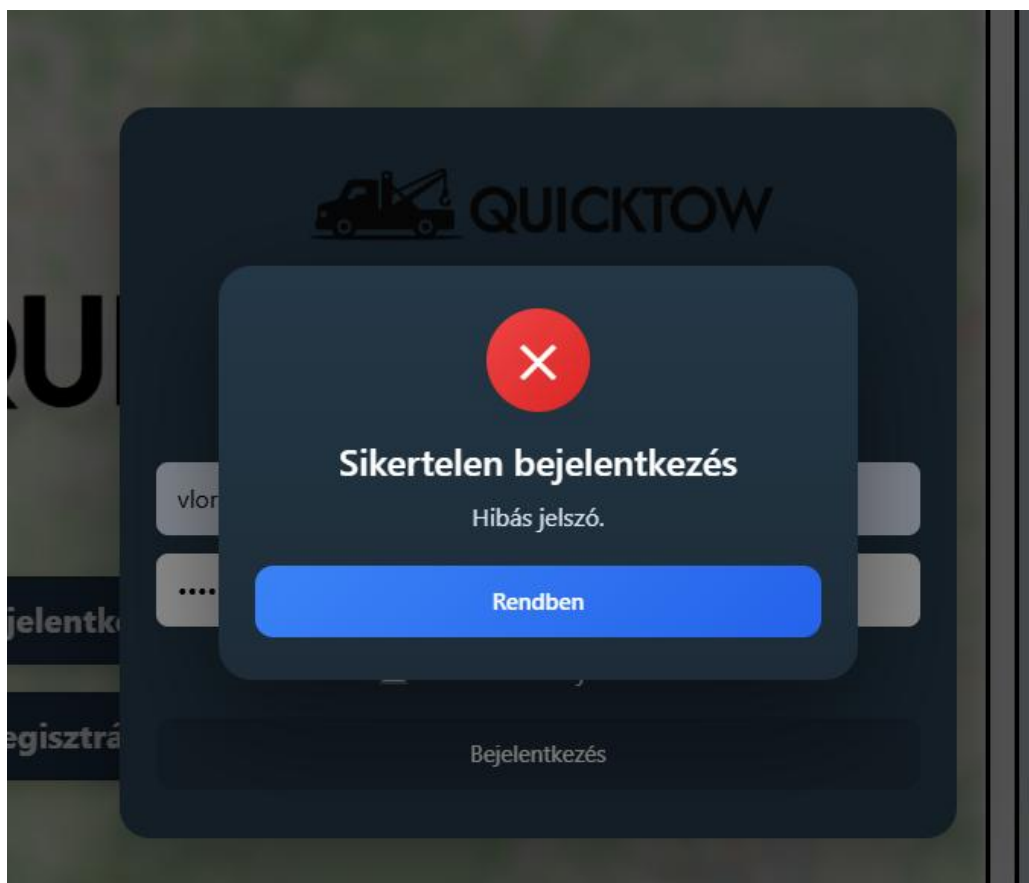
Regisztráció vagy adatok módosítása esetén a hiányzó vagy hibás adatot felugró ablak jeleníti meg.



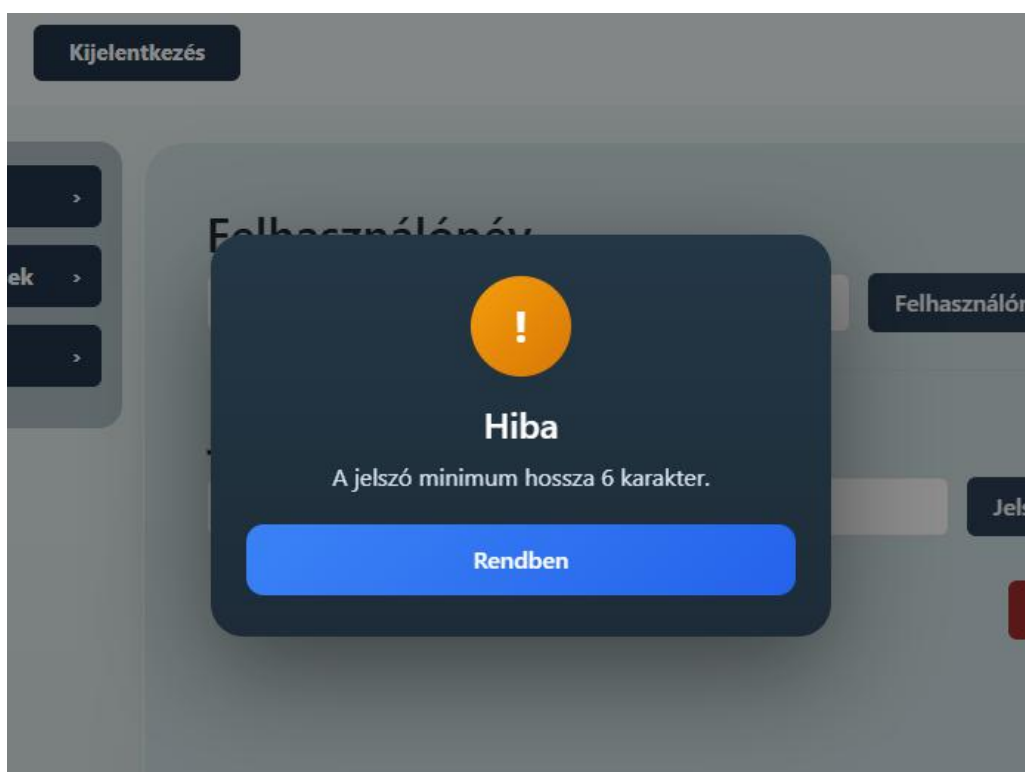
77. Hiányzó adat regisztráció esetén



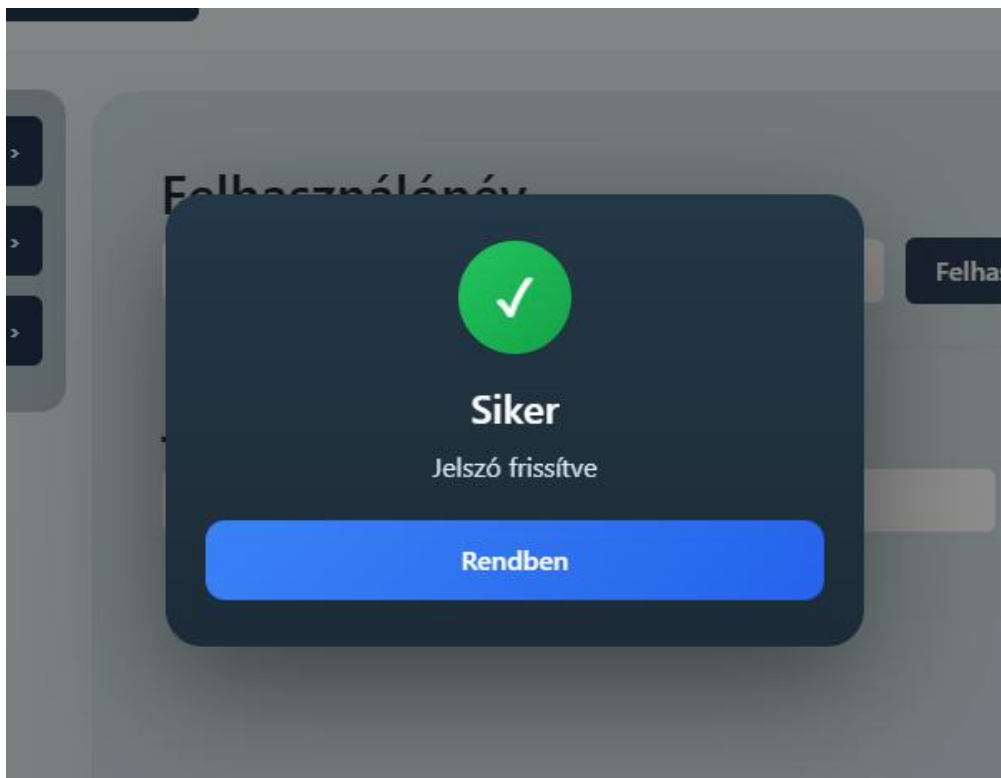
78. Érvénytelen e-mail cím regisztráció közben



79. Hibás jelszó bejelentkezésnél



80. Hiba jelszó változtatása közben



81. Sikeres jelszó változtatás

Keresésnél vagy kérés küldésénél a gombok csak akkor válnak kattinthatóvá, ha a szükséges adatok meg vannak adva.



82. Keresés gomb adatok megadása előtt

Vontatási Kérés

Autó kiválasztása:



Renault Thalia

Kiválasztva

ABC-123

Autó törlése

+

Új autó hozzáadása

Tartózkodási hely pontosítása (az autómentő számára):

Sáta Széchenyi István utca 99

Lerakási cím (utca) megadása:

pl. Ózd, Március 15 utca / Ózd, Göngyösi Gumi Kft

Cím keresése

Végleges Ár: - Ft

- Km

2222 Ft / Km

Mégsem

Kérés küldése

83. Kérés küldése gomb nem kattintható az adatok megadása előtt.

Vontatási Kérés

Autó kiválasztása:



Renault Thalia

Kiválasztva

ABC-123

Autó törlése

+

Új autó hozzáadása

Tartózkodási hely pontosítása (az autómentő számára):

Sáta Széchenyi István utca 99

Lerakási cím (utca) megadása:

Ózd, malom út

Cím keresése

Végleges Ár: 64438 Ft

29 Km

2222 Ft / Km

Mégsem

Kérés küldése

84. Kérés küldése gomb az adatok megadása után

8. Továbbfejlesztési lehetőségek

A webalkalmazás egyik legfontosabb továbbfejlesztési lehetősége a Leaflet és OpenStreetMap (OSM) térkép leváltása valamilyen erősebb (pl. Google Maps) térképre, mivel az OSM-en sok helyről hiányoznak a házszámok, így pontos cím keresésére sokszor nincs lehetőség.

Másik fontos lehetőség a valós idejű frissítés (webhookok) használata. Jelenleg minden adat ami változik és adatbázisból frissül a weboldalon, intervalokban, meghatározott időn belüli lekérésekkel működik. Ennek hatásosabb megoldása a webhookok használata lenne, így mindig csak akkor kéne lekérni backendből ezeket az adatokat ha változtak.

Egyéb továbbfejlesztési lehetőségek:

- az autómentőket kilistázó komponensben szűrés vagy rendezés ár / km szerint is
- több fajta szolgáltatás közül választás, például segédindítás (bebikázás), kerékcseré, üzemanyag szállítás, stb.
- alkalmazáson keresztüli fizetés az egyszerűség és átláthatóság érdekében
- különböző autómentő típusok és méretkorlátok nagyobb járművek mentésére

A továbbfejlesztés során érdemes lehet mobilalkalmazás készítése is, amely még gyorsabb és stabilabb felhasználói élményt biztosítana.

9. Irodalomjegyzék

- <https://angular.dev/>
- <https://laravel.com/docs>
- <https://leafletjs.com>
- <https://www.openstreetmap.org>
- <https://nominatim.org/release-docs/latest/>
- <https://project-osrm.org>
- <https://stackoverflow.com/questions>